Ariel Lai
Ivan Zheng
Rumman Jan

Digital Galton Board

Contents

1 Introduction **3**

2 Design and Testing Methods **4**
2.1 Hardware Setup and Schematic 4
2.2 Original Working Software 5
2.2.1 Board geometry and ball lifecycle 5
2.2.2 Original collision physics 6
2.2.3 Original frame loop and display 7
2.2.4 Histogram and User Interface 7
2.2.5 Collision audio 8
2.3 Optimizations 9
2.3.1 Local Peg Lookup 9
2.3.2 Using Both Cores 9
2.3.3 Distance Approximation 10
2.3.4 Compact Ball Storage and Temporary PhysicsBall 10
2.3.5 Initial Overclocking 11
2.3.6 Reducing Drawing Work 11
2.3.7 Caching the Static Background 11
2.3.8 Monochrome Framebuffers 12
2.3.9 Further Overclocking with Increased Core Voltage 13
2.3.10 Balancing Physics and Drawing Between Cores 13
2.3.11 Background Restoration 13
2.3.12 Removing Temporary PhysicsBall Conversions 14
2.4 Final Integrated Software 14
2.4.1 Frame sequence and concurrency 14
2.4.2 Optimization Summary 15
2.5 Testing Methods 16
2.5.1 Hardware, Controls, and Display 16
2.5.2 Physics, audio, and distribution 16
2.5.3 Timing, multicore, DMA, and memory 17
2.6 AI Use 17
2.7 Optimization Progression 18
2.8 Memory and Timing 19
2.9 Display and Demonstration Evidence 20
2.10 Distribution and Numerical Limitations 21

3 Conclusion **22**

4 Answers to the Lab Questions **22**
4.1 CLT Assumptions in the Physical Model 22
4.2 Why More Rows Improve the Approximation 23
4.3 Ideal 16-Row Mean and Standard Deviation 24

A Code Appendix **25**
A.1 Main Program: `galton_board.c` 25
A.2 VGA Driver: `VGA/vga16_graphics_v3.c` 43
A.3 VGA Header: `VGA/vga16_graphics_v3.h` 63
A.4 Horizontal Sync: `VGA/hsync.pio` 65
A.5 Vertical Sync: `VGA/vsync.pio` 66
A.5.1 Monochrome Output: `VGA/rgb.pio` 67

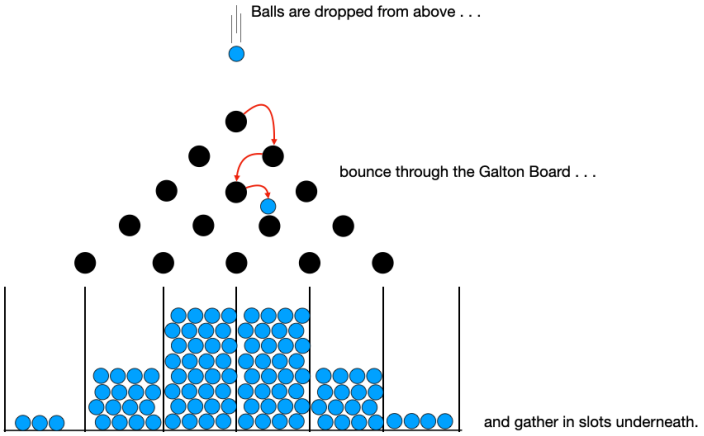


Figure 1: Illustration of a Galton Board

1 Introduction

A Galton Board consists of several staggered rows of pegs. When a ball is dropped from the top, it bounces off the pegs as it moves downward and eventually lands in one of the bins at the bottom. Ideally, each time a ball hits a peg, it has a 50% chance of moving left or right. As more balls are dropped, the number of balls collected in each bin follows a binomial distribution, which begins to resemble a Gaussian distribution as the number of rows increases, as explained by the central limit theorem.

For this project, we built a digital Galton Board using the Raspberry Pi Pico 2 (RP2350), with 16 rows of pegs, ball collision physics, and a histogram displayed on a VGA screen. The project was divided into three weeks, with each week building on the previous one. In the first week, we started by implementing the rotary encoder, simulating a single ball bouncing off a peg under gravity, and adding sound effects using DMA whenever a collision occurred. In the second week, we expanded the board to 16 rows of pegs and added multiple balls. We also implemented a histogram to track where the balls landed, along with VGA statistics showing the number of active balls, total balls dropped, and time since boot. The rotary encoder was used to adjust the number of balls. In the third week, our main goal was to maximize the number of balls we could simulate while maintaining 60 fps. We also added controls for adjusting bounciness and gravity, updated the VGA statistics, and implemented an LED indicator to detect missed frame deadlines.

Initially, our implementation could only handle around 500–600 balls before running into performance issues. We spent the final part of the project trying different ways to reduce the time spent updating and drawing the balls. As discussed in Section 2.3, we made several changes to improve performance and eventually reached a reported capacity of 22,200 balls at the 60 fps target. The optimization progression, memory use, and limitations of the recorded results are discussed in Sections 2.7–2.10.

2 Design and Testing Methods

2.1 Hardware Setup and Schematic

Our hardware consisted of a Raspberry Pi Pico 2, a VGA monitor, a rotary encoder with a built-in push button, and an MCP4822 DAC connected to the audio output. Figure 2 shows the connections, and Figure 3 shows the assembled setup. We connected the encoder's A and B signals to GPIO 15 and 14, respectively, and used GPIO 22 and 26 for the push-button pull-up and detection arrangement. These inputs allowed the user to adjust the simulation parameters and cycle between them.

For VGA output, GPIO 16 and 17 supplied horizontal and vertical synchronization. GPIO 18 and 19 connected to the green channel through 470 Ω and 330 Ω resistors, respectively, while GPIO 20 and 21 connected to the blue and red channels through 330 Ω resistors. These resistors limited current and, together with the monitor's input termination, reduced the GPIO output voltages to analog video signal levels. The two green connections provided different contributions to the green signal in the original color display. We retained this resistor network after changing the display to monochrome. The PIO state machines generated the synchronization and pixel signals, while DMA supplied framebuffer data without requiring the CPU to transmit each pixel.

For audio, we connected the MCP4822 DAC's chip-select, clock, and serial data inputs to GPIO 5, 6, and 7, respectively. The DAC was powered from 3.3 V, with a 0.1 μF bypass capacitor between its supply and ground to reduce supply noise. Its active-low LDAC input was tied to ground, allowing the selected analog output to update when each valid SPI write completed without requiring a separate latch pulse. Channel A's output connected to the tip of the 3.5 mm audio jack, and the sleeve connected to ground; the ring and DAC channel B were unused. The jack fed a powered speaker, and the connected components shared a common ground. We also used the Pico's onboard LED, controlled through GPIO 25, to indicate when the measured frame work exceeded the 60 fps deadline.

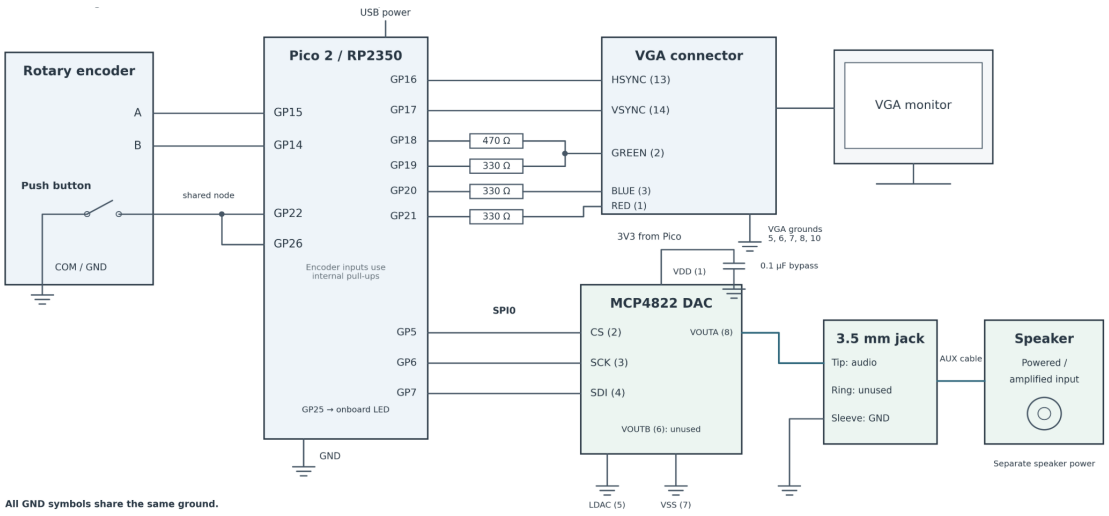


Figure 2: Wiring schematic for the Pico 2, VGA interface, rotary encoder, and audio circuit.

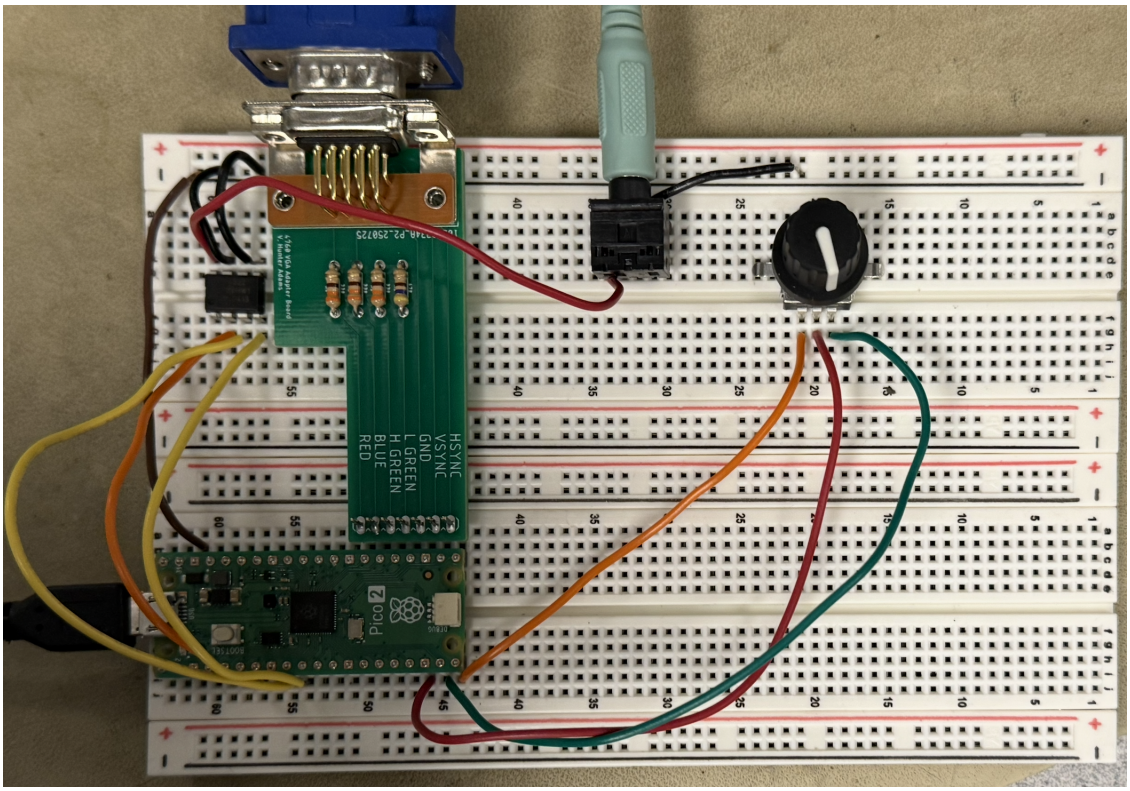


Figure 3: Physical hardware setup and connected peripherals.

2.2 Original Working Software

We began with a single-ball, single-peg implementation to establish collision physics, animation, and audio playback. We then expanded it into a complete 16-row Galton Board with a landing histogram and interactive controls. Before the optimizations described in Section 2.3, this implementation supported approximately 550 balls at 60 frames per second. The following subsections describe its geometry, physics, display, controls, and audio.

2.2.1 Board geometry and ball lifecycle

We first implemented a single ball falling onto a stationary peg on the 640×480 VGA display. The ball had a radius of 4 pixels, and the peg had a radius of 6 pixels and was positioned at (320, 80). The ball spawned at (320, 0) with zero vertical velocity and a randomized horizontal velocity between -0.5 and 0.5 pixels per update. Gravity increased its downward velocity by 0.37 each update, while a bounciness factor of 0.5 reduced its velocity after a collision. The ball reflected off the top and side boundaries and respawned when it passed below the screen. We then expanded this setup into a triangular board with 16 rows and 136 pegs. Using nested loops, we placed one peg in the first row and added one peg to each successive row. Each row was centered horizontally, with 38 pixels between neighboring pegs and 19 pixels between rows. This staggered adjacent rows by half the horizontal spacing, creating the branching arrangement associated with Pascal's triangle: in the ideal model, the number of paths reaching a bin is the sum of the paths arriving from its two neighboring

positions above. The completed board had 17 landing bins. Each ball's horizontal position at the bottom selected the histogram bin to increment before the ball respawned.

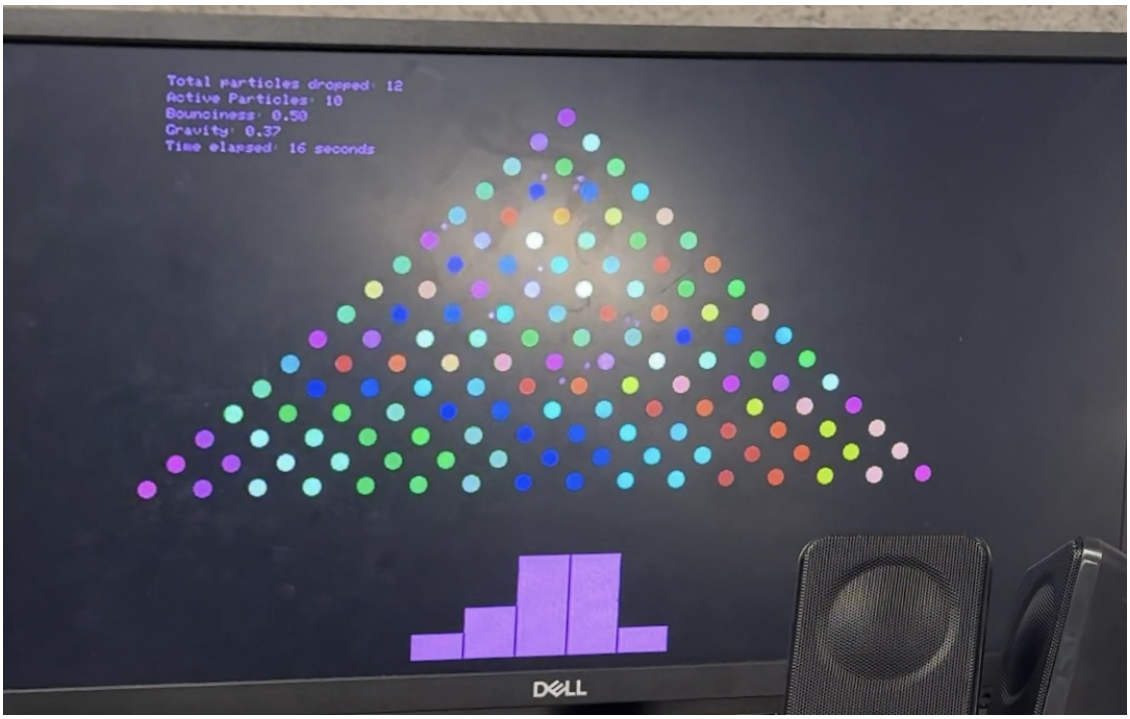


Figure 4: Original Galton Board implementation.

2.2.2 Original collision physics

For each ball, we first updated its position by adding its horizontal and vertical velocity components. We then checked for collisions by looping through the pegs and calculating the horizontal and vertical displacement between each peg and the ball. To avoid unnecessary square-root calculations, we first checked whether both displacement magnitudes were less than the combined ball and peg radii, or 10 pixels. For candidates passing this check, we calculated $d = \sqrt{dx^2 + dy^2}$ and identified a collision when $d < 10$.

To resolve a collision, we calculated the outward normal $\mathbf{n} = (dx/d, dy/d)$ and moved the ball 11 pixels from the peg center along this direction, leaving one pixel of clearance. If the centers coincided, we used an upward normal to avoid dividing by zero. We then calculated $\mathbf{v} \cdot \mathbf{n}$ to determine whether the ball was approaching the peg. When this value was negative, we reflected its velocity according to

$$\mathbf{v}_{\text{reflected}} = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}.$$

Both velocity components were then multiplied by the bounciness factor, initially 0.5, and a collision sound was triggered. Finally, we handled boundary collisions, respawned balls that crossed the bottom, and applied gravity to the remaining balls.

The simulation used discrete position updates, stationary pegs, and no ball–ball collisions. Each ball was processed independently, so checking every ball against all 136 pegs became

increasingly expensive as the ball count grew. This full search was one of the main targets of our later optimizations.

2.2.3 Original frame loop and display

The original full-board version handled approximately 500–600 balls. Each frame cleared the drawing buffer, updated the balls, and drew the scene again. Clearing the screen also erased the stationary pegs, so we had to redraw them along with the balls, histogram, and statistics. The balls were drawn as filled circles, which meant writing several pixels for every ball on every frame.

The display originally used four bits per pixel to support 16 colors. At 640×480 pixels, each framebuffer occupied 153,600 bytes, and two framebuffers required 307,200 bytes. This was a large memory cost before accounting for the ball array and audio samples. Meanwhile, checking every ball against 136 pegs and repeatedly drawing filled shapes limited how much work we could finish within the 16.67 ms budget for 60 fps. Our later changes addressed both the physics work and the cost of rebuilding the display.

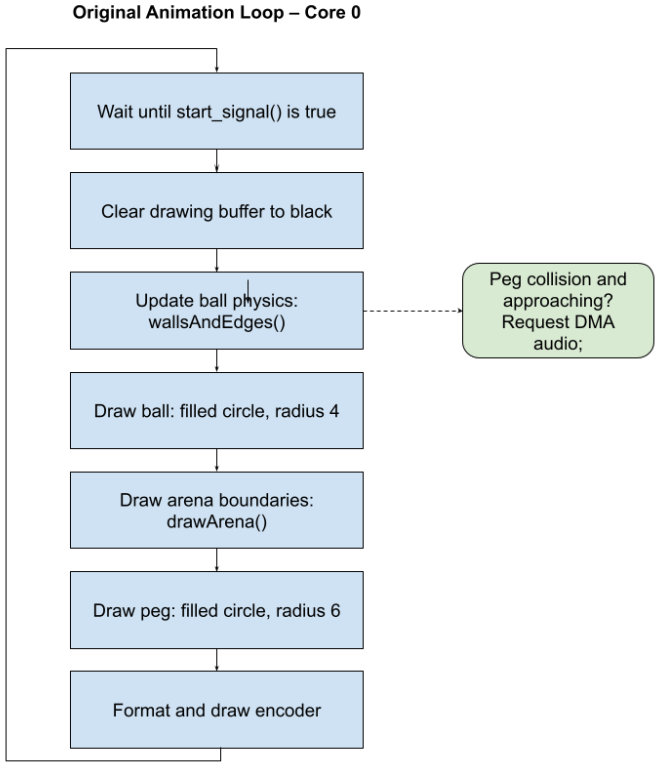


Figure 5: Frame-processing sequence in the original implementation.

2.2.4 Histogram and User Interface

For the histogram, we followed the demonstration video provided on the course webpage to model how the balls accumulated in the bins at the bottom of the Galton Board. Since our

board had 16 rows of pegs, we divided the landing area into 17 bins. Whenever a ball reached the bottom, we used its x-coordinate to determine which bin it landed in and increased the corresponding count before respawning the ball at the top. We also normalized the height of each histogram bar relative to the bin with the highest count so that the distribution would fit within the available space on the VGA display, even as more balls were dropped.

For the user interface, we used a rotary encoder to control the number of balls, bounciness, and gravity. We implemented GPIO interrupts to detect changes in the encoder's A and B signals, allowing us to determine the direction of rotation without continuously polling the encoder. We also used the built-in push button to switch between the three parameters. To avoid registering a single press multiple times, we added button debouncing by requiring three consecutive matching readings before accepting a change.

The push button was polled with a requested interval of 10 ms, and three consecutive matching readings were required to confirm a change. With regularly spaced polls, this gives approximately 20–30 ms of debounce latency; cooperative scheduling can increase the actual interval. This filtering trades a short response delay for rejection of switch bounce. A confirmed press advanced the parameter selection once, while holding the button did not repeatedly cycle through the parameters.

We limited the number of balls to the maximum capacity supported by the implementation at each stage and kept bounciness and gravity between 0 and 1. On the VGA display, we showed the number of active balls, total balls dropped, time since boot, and the current values of bounciness and gravity. The selected parameter was also highlighted so the user could see which value they were changing. Whenever a parameter was adjusted, we reset the histogram and total number of balls dropped so that the distribution would start over with the new settings.

2.2.5 Collision audio

To provide audible feedback for peg collisions, we created a short "thunk" by generating a 400 Hz sine wave with an amplitude that decreased linearly to zero. We calculated the waveform once during initialization and stored 2,205 samples, corresponding to 100 ms at the nominal sampling rate of 22,050 samples per second. Each sample was shifted around the DAC's midpoint of 2048 and combined with the control value `0x3000`. In the resulting 16-bit word, bit 15 selected channel A, bit 14 was unused, bit 13 selected unity gain, and bit 12 enabled the output. Bits 11–0 contained the 12-bit sample value. With the DAC's internal 2.048 V reference and unity gain, one count represented an ideal 0.5 mV step, and the midpoint corresponded to 1.024 V. SPI0 used 16-bit, MSB-first transfers in mode 0, with a requested clock rate of 20 MHz.

We used two DMA channels to play the sample array. The control channel restored the sample buffer's starting address and then triggered the data channel, which transferred samples to the SPI peripheral. A DMA timer paced these transfers at approximately 22,050 samples per second; the SPI clock determined how quickly each individual word was transmitted. In the final implementation, each core recorded whether its physics updates detected an approaching peg collision. After both cores finished, core 0 checked these requests and attempted to start playback. If the previous sound was still playing, the new request was skipped. Multiple collisions within a frame therefore produced a single playback request

rather than separate sounds for every impact. This provided collision feedback while allowing the CPU cores to continue physics and drawing without transmitting every audio sample themselves.

2.3 Optimizations

The original full-board implementation supported approximately 550 balls. We improved the implementation incrementally, addressing processing time, drawing overhead, and memory usage as each became a limiting factor. Each ball count below represents the total capacity reached at that stage.

2.3.1 Local Peg Lookup

The first thing we chose to optimize, as recommended by the professor, was to avoid checking all 136 pegs for every ball during collision detection. In the original implementation, we looped through every peg and calculated the distance between the ball and each peg to check for a possible collision. Since the pegs were arranged in evenly spaced rows, we could instead use the ball's y-coordinate to find the nearest row and its x-coordinate to find the nearest peg within that row.

To implement this, we first calculated the row using the ball's vertical position and the spacing between rows:

```
int row = (ball->y - pegs[0].y + row_spacing / 2)
          / row_spacing;
```

Since each row contains one more peg than the previous row, we used the triangular indexing formula to find the starting index of that row. We then calculated the nearest column using the ball's x-coordinate:

```
int first = row * (row + 1) / 2;

int col = (ball->x - pegs[first].x + col_spacing / 2)
          / col_spacing;
```

After finding the nearest peg, we checked whether the row and column were valid and whether the ball was close enough in both the x and y directions before performing the actual collision calculation. This meant that instead of checking all 136 pegs for every ball, we only needed to check at most one peg. By reducing the number of collision checks, we increased the number of balls we could simulate from approximately 550 to 1,500.

2.3.2 Using Both Cores

After local peg lookup brought the capacity to approximately 1,500 balls, we split the physics updates between the two cores. Each core worked on a separate range of the ball array. Since balls did not collide with each other, updating one ball did not depend on another ball's

position or velocity. Both cores could therefore calculate motion and peg collisions at the same time.

In this first multicore version, core 0 finished its own updates and waited for core 1 before drawing any balls. Waiting ensured that core 0 read the completed positions from core 1's range. Only core 0 drew to the framebuffer. This increased capacity to approximately 3,000 balls. Later, we reduced the waiting time by letting core 0 draw its own balls while core 1 was still working, as described in the workload balancing section.

2.3.3 Distance Approximation

After using both cores brought our capacity to approximately 3,000 balls, we targeted the square-root calculation in the collision code. We replaced the distance calculation used to determine the collision normal with the alpha-max plus beta-min approximation:

$$d_{\text{approx}} = \max(|dx|, |dy|) + \frac{1}{2}\min(|dx|, |dy|).$$

We chose $\alpha = 1$ and $\beta = 1/2$ because the calculation only requires comparisons, addition, and a right bit shift instead of a square root. For example, if $dx = 3$ and $dy = 4$, the actual distance is $\sqrt{3^2 + 4^2} = 5$, while our approximation gives $4 + 3/2 = 5.5$. This is a 10% overestimate of the distance.

We kept the actual collision detection accurate by comparing $dx^2 + dy^2$ with the squared sum of the ball and peg radii. Therefore, the approximation only affected how we calculated the collision normal and resolved the collision, rather than whether an overlap was detected.

Since the approximate distance is not always equal to the true distance, the resulting normal is not exactly a unit vector. This can slightly change the reflected velocity and the position correction after a collision. We accepted this tradeoff because our goal was to simulate as many balls as possible while keeping the motion visually reasonable. With this change, our capacity increased from approximately 3,000 to 5,600 balls.

2.3.4 Compact Ball Storage and Temporary PhysicsBall

At 5,600 balls, we reduced memory usage by changing how each ball was stored. Originally, a ball had four 32-bit fixed-point fields: two for position and two for velocity. Each field had 15 fractional bits, and the structure occupied 16 bytes. We kept both positions in that format but changed the stored velocities to 16-bit values with eight fractional bits. This brought the structure down to 12 bytes.

To keep the existing physics code working, we used a temporary `PhysicsBall` with all four fields in the original 32-bit format. For each update, we copied the ball into this structure and rescaled its velocities. After movement, collisions, and gravity, we rounded the velocities back to the compact format and stored the result. There was no second array of expanded balls; each update used only a temporary working copy.

At the same count of 5,600 balls, the array decreased from 89,600 to 67,200 bytes, saving 22,400 bytes, or 25%. The compiled `data + bss` total decreased from 408,944 to 386,552 bytes. The net reduction was eight bytes smaller than the array saving because the revised build included additional diagnostic storage. Position resolution stayed at 1/32768 pixel,

while velocity resolution became 1/256 pixel per update. The memory savings came with less velocity precision and extra conversion work on every update, which we addressed later.

2.3.5 Initial Overclocking

After the distance approximation increased our capacity to approximately 5,600 balls, we increased the processor clock to complete more work within each frame. Overclocking runs the processor above its standard clock frequency, providing more clock cycles within the approximately 16.67 ms available for a 60 fps update. This allowed us to process more balls without changing the collision algorithm.

We changed the system clock through the Pico SDK's `set_sys_clock_khz()` function and increased the frequency in stages. Our total capacity reached approximately 7,500 balls at 200 MHz, 9,700 at 250 MHz, and 11,000 at 280 MHz. Because peripheral timing also depends on the system clock, we adjusted the PIO clock dividers and DMA audio pacing to preserve the VGA timing and audio sample rate as the processor frequency increased.

At this stage, we had not increased the core supply voltage and could not continue increasing the clock successfully beyond 280 MHz. We therefore stopped overclocking temporarily and focused on reducing the amount of drawing work performed each frame. The later voltage increase and additional overclocking are discussed separately.

2.3.6 Reducing Drawing Work

Another optimization we made was reducing the amount of drawing required for each frame. Initially, we were using the `fillCircle()` function from the VGA graphics demo to draw the balls and pegs. However, filling these shapes required writing to more pixels, which became expensive as we increased the number of balls. To reduce this overhead, we changed the pegs from filled circles to outlines using `drawCircle()`, and similarly changed the histogram bars from filled rectangles to outlines using `drawRect()`.

We also reduced the size of the balls. Initially, each ball was drawn as a filled circle with a radius of 4 pixels. We replaced this with a much simpler 2×2 pixel square using `fillRect()`. This reduced the number of pixels we needed to draw for each ball and avoided the additional calculations involved in drawing a circle. Although the balls appeared smaller and less circular, they were still visible as they moved through the board. Although the balls appeared smaller and less circular, they were still visible as they moved through the board.

Together, these changes reduced the drawing overhead and allowed us to increase the total number of balls to approximately 13,000. At this point, available RAM became the next limitation preventing us from increasing the ball count further.

2.3.7 Caching the Static Background

Although the balls moved every frame, the peg positions remained unchanged. We were therefore repeating the same peg-drawing work each time we cleared the screen. To avoid this, we drew the pegs once onto a black background and saved the resulting image in a separate buffer. At the beginning of each frame, we copied this cached background into the drawing buffer, removing the previous ball positions and restoring the pegs in one operation.

We then drew the updated balls, histogram, and statistics over the restored background. These elements remained dynamic because their positions or values changed during the simulation. Caching required additional memory for the background image, but replaced repeated calls to the peg-drawing routines with a buffer copy. This also provided the basis for our later optimization of using DMA to restore the background while the CPU cores calculated ball physics.

2.3.8 Monochrome Framebuffers

After reducing the drawing work, we reached approximately 13,000 balls and became limited by memory. The ball array occupied 156,000 bytes, while the two VGA framebuffers together required 307,200 bytes. Our original display stored four bits per pixel to support 16 colors, but we no longer needed those colors for the board. We therefore changed the framebuffers to one bit per pixel, with each bit representing either black or white. This packed eight pixels into each byte instead of two and reduced each 640×480 framebuffer from 153,600 to 38,400 bytes.

To support the new format, we updated the drawing routines to locate the byte containing a pixel and set or clear its corresponding bit without changing neighboring pixels. We also modified the RGB PIO program to read these bits in least-significant-bit order and output black or white on the existing VGA color pins. The display resolution remained unchanged.

Table 1: Static RAM allocation before and after monochrome conversion, with the ball array held at 13,000 balls.

Allocation	**Before (bytes)**	**After (bytes)**
Balls	156,000	156,000
VGA buffer 0	153,600	38,400
VGA buffer 1	153,600	38,400
Cached peg background	38,400	38,400
Audio samples	4,410	4,410
Other static allocations	8,778	7,754
Total data + bss	**514,788**	**283,364**

As shown in Table 1, the conversion reduced framebuffer storage by 75%, freeing 230,400 bytes across the two buffers. We calculated the pre-conversion value for other static allocations by subtracting the listed allocations from the reported total. Changes in those allocations account for the additional 1,024-byte reduction in total static memory. The cached peg background was already 38,400 bytes, so its allocation did not change.

The smaller framebuffers also reduced the amount of data needed for framebuffer transfers and copies. Although we lost the ability to display multiple colors, the additional memory allowed us to increase the ball count and take fuller advantage of the earlier rendering improvements. Our total capacity then reached approximately 14,700 balls.

2.3.9 Further Overclocking with Increased Core Voltage

After the monochrome conversion relieved the memory constraint, we returned to overclocking to increase the amount of computation we could complete within each frame. Our earlier attempts had stopped at 280 MHz without increasing the core voltage. This time, we raised the core regulator setting to 1.35 V before increasing the processor clock, allowing us to explore higher frequencies.

We set the regulator voltage before changing the system clock and allowed a short delay for the voltage to settle. We then increased the processor clock to 380 MHz, adjusting the VGA clock dividers and audio pacing to preserve their timing. With the earlier rendering and memory improvements already in place, this stage brought our total capacity to approximately 19,000 balls.

2.3.10 Balancing Physics and Drawing Between Cores

We also tried to improve performance by changing how we distributed the work between the two cores. Initially, we had divided the physics calculations equally, so each core was responsible for updating 50% of the balls. However, we realized that core 0 was doing more work since it also had to draw all the balls on the VGA display, while core 1 was only responsible for updating their positions. This meant that an equal split was not necessarily the most efficient way to use both cores.

We decided to give core 1 a larger portion of the physics calculations, changing the split from 50:50 to 30:70. Core 0 now updated 30% of the balls, while core 1 handled the remaining 70%. We also changed the implementation so that core 0 did not have to wait for core 1 to finish all its calculations before starting to draw. Instead, once core 0 finished updating its own balls, it immediately started drawing them while core 1 continued working on the rest.

We implemented this using the following code:

```
int split = num_balls * 3 / 10;
struct PhysicsJob job = {split, num_balls};
```

Core 0 only waited for core 1 to finish before drawing the remaining 70% of the balls, making sure that their positions had already been updated. This allowed us to overlap some of the drawing and physics calculations instead of performing them one after another. By distributing the work more efficiently, we reduced the time spent waiting between the two cores and improved the overall performance of the simulation.

2.3.11 Background Restoration

Once we had a cached monochrome background, core 0 restored it by using `memcpy()` to copy the 38,400-byte background into the drawing buffer before either core began physics. This removed the previous ball positions and restored the stationary pegs without drawing each peg again.

The background copy completed before physics and drawing began, so it did not overlap with those operations. Its execution time was included in our frame-time measurement. The background remained in RAM, so caching traded additional memory for less repeated drawing work.

2.3.12 Removing Temporary PhysicsBall Conversions

The temporary `PhysicsBall` let us save memory without rewriting the physics immediately, but it added copying and conversion work for every ball. We later changed the calculations to use the compact velocities directly, removing the need to expand the entire ball into a temporary structure and copy it back after each update.

Positions stayed as 32-bit values with 15 fractional bits. Velocities stayed as signed 16-bit values with eight fractional bits, giving a resolution of 1/256 pixel per update. Multiplication still used wider intermediate values, and calculations had to account for the different position and velocity scales before storing their results. The ball structure therefore remained 12 bytes.

This reduced repeated conversion work while keeping the earlier memory savings. It also retained the compact format's rounding and range limits, which could affect individual trajectories. With this change and the earlier optimizations, the reported capacity reached approximately 22,200 balls.

2.4 Final Integrated Software

2.4.1 Frame sequence and concurrency

In our final implementation, we overlapped physics calculations and drawing to reduce the time spent waiting between tasks. At the beginning of each frame, core 0 restored the cached background using `memcpy()` before either core began updating the balls. We divided the physics calculations in a 30:70 ratio, with core 0 handling 30% and core 1 handling 70%. We chose this split because core 0 was also responsible for drawing the balls and managing the VGA display, while core 1 mainly performed physics calculations.

For example, with 10,000 balls, core 0 updated 3,000 balls while core 1 updated the remaining 7,000. Once core 0 finished updating its own balls, it immediately started drawing them because the background copy had already completed. While core 0 was drawing its balls, core 1 could continue updating its assigned range.

After drawing its first group of balls, core 0 waited for core 1 to finish its physics calculations before drawing the remaining balls. This ensured that core 0 always used updated ball positions rather than reading values that core 1 was still modifying. Only core 0 wrote to the drawing framebuffer, so the two cores did not attempt to draw to the same memory simultaneously.

The histogram and statistics were updated after the necessary ball calculations were completed, and the drawing buffer was presented through the VGA driver's buffer-swapping mechanism. Collision sounds were handled separately through DMA, allowing audio playback to continue without requiring the processor to transmit every sample. All of this work, including background restoration and synchronization delays, was measured against the approximately 16.67 ms frame budget targeted for 60 fps.

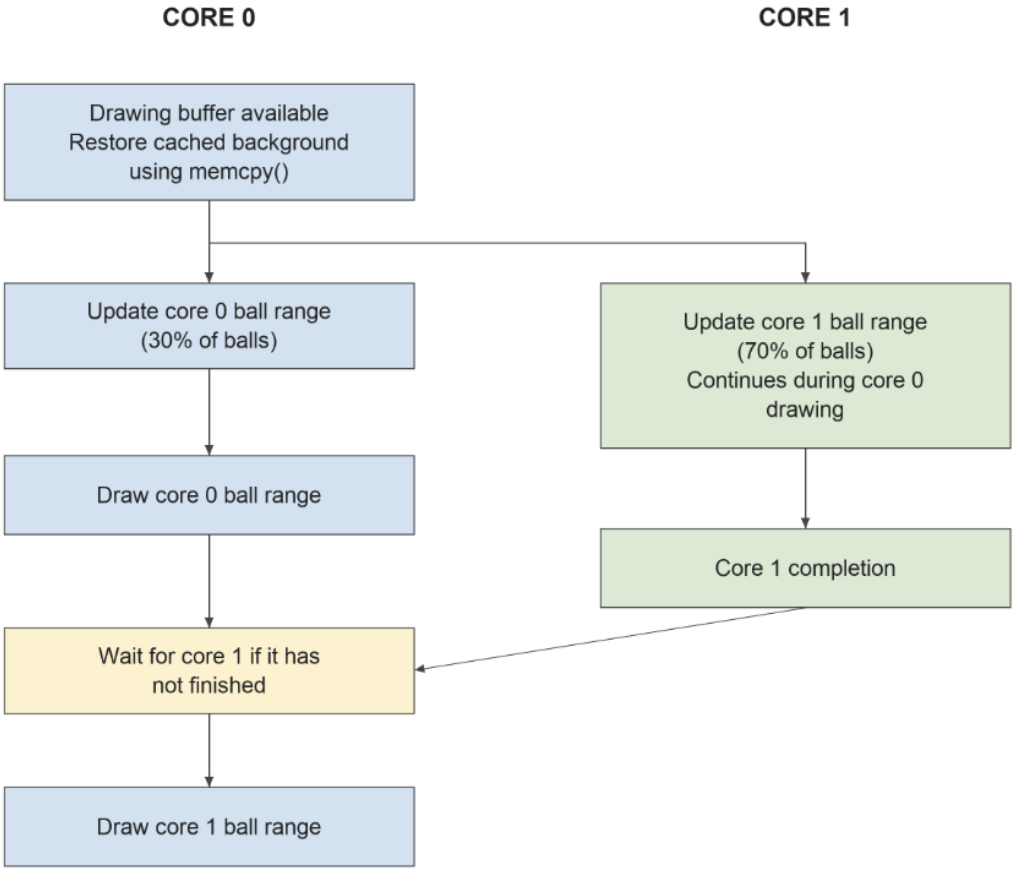


Figure 6: Frame schedule showing physics on both cores, background DMA, and drawing on core 0.

2.4.2 Optimization Summary

Overall, we increased the number of balls from approximately 550 to 22,200 by reducing collision calculations, improving how both cores were used, simplifying drawing, and reducing memory usage.

Table 2 summarizes our optimizations and the total ball count achieved at each stage.

Table 2: Summary of optimizations

Optimization	**Change**	**Balls**
Baseline	Full peg search and drawing	550
Local peg lookup	Checked only the nearest peg	1,500
Dual-core physics	Split physics across both cores	3,000
Distance approximation	Avoided square-root calculations	5,600
Compact storage	Reduced ball size from 16 to 12 bytes	5,600
Initial overclocking	Increased clock to 280 MHz	11,000
Drawing optimization	Simplified ball, peg, and histogram drawing	13,000
Cached background	Reused pre-drawn peg background	–
Monochrome buffers	Reduced framebuffer memory by 75%	14,700
Further overclocking	Increased clock to 380 MHz	19,000
Core balancing	Changed physics split to 30:70	–
Direct compact physics	Removed temporary ball conversions	22,200

2.5 Testing Methods

2.5.1 Hardware, Controls, and Display

We tested the VGA display, rotary encoder, and push button to make sure the hardware and user controls worked as expected. Initially, the VGA output had some glitches and appeared very dim. After making changes to the display implementation, we were able to get the output working properly. We also checked the rotary encoder by turning it in both directions and confirmed that it correctly increased or decreased the selected parameter. The push button also worked as expected, allowing us to cycle between the different parameters.

We visually checked the placement of all 16 rows of pegs and confirmed that the balls were bouncing off them correctly, including after implementing the local peg lookup optimization. We also checked that the smaller balls and histogram outlines were still clearly visible on the VGA display.

During testing, we noticed that the histogram was not filling correctly and that the balls were starting too low on the screen. We went back through the ball positions, velocities, and related calculations to identify the issues and correct the behavior. After these changes, the histogram and displayed statistics updated as expected. We also checked the hardware connections and did not encounter any wiring issues.

2.5.2 Physics, audio, and distribution

We began testing with one ball and one peg so that we could follow individual collisions. We checked that the ball accelerated downward, bounced away from the peg, and respawned after leaving the bottom of the screen. We also checked its response to the top and side boundaries. After expanding to the full board, we adjusted gravity and bounciness through the rotary encoder and observed the resulting motion. Increasing gravity produced faster downward acceleration, while increasing bounciness allowed balls to retain more speed after collisions.

After introducing the distance approximation, we visually checked that balls continued to bounce away from pegs and watched for sticking, overlap, or unexpected changes in direction. These checks assessed whether the motion remained plausible rather than measuring the numerical error of individual collisions.

For audio, we listened for the sound as the single ball struck the peg, making it easier to associate playback with a collision. We then checked that collision audio continued during full-board operation. At higher ball counts, overlapping collisions could not be distinguished individually by listening, so this check confirmed audible feedback rather than a separate sound for every impact.

We tested the histogram by observing bin counts accumulate as balls passed below the board and checking that the bars remained within the available display area. Over repeated landings, we observed a central peak with fewer balls reaching the outer bins, consistent with an approximately bell-shaped distribution. This was a qualitative check of the displayed distribution; it did not establish a numerical fit to a Gaussian.

2.5.3 Timing, multicore, DMA, and memory

We tracked the supported ball count as we changed the physics, rendering, and clock settings. At 60 fps, each frame has approximately 16.67 ms to finish. In the final implementation, frame-work timing started after the drawing-buffer availability signal and included background restoration using `memcpy()`, control updates, physics, synchronization, ball drawing, histogram drawing, and statistics drawing. It excluded the preceding wait for buffer availability. The software compared this duration with an integer threshold of 16,666 μs, incremented a missed-frame counter when the threshold was exceeded, and drove the onboard LED high for that frame. GPIO 2 bracketed the frame work, while GPIO 3 bracketed the combined physics and ball-drawing routine. These signals provided instrumentation for checking processing time, although we did not retain final timing measurements.

We checked memory use through compiled `data + bss` totals and the sizes of the main arrays. Comparing builds at the same ball count let us separate storage savings from changes in capacity. Compact ball storage reduced the total from 408,944 to 386,552 bytes at 5,600 balls, and monochrome framebuffers reduced it from 514,788 to 283,364 bytes at 13,000 balls. These totals cover static allocations; the program also needs room for both cores' stacks and any dynamic allocations. Our notes do not record the final run duration or deadline-miss count.

2.6 AI Use

AI assistance was used to explain technical concepts, suggest debugging approaches, and discuss ways to write and implement code for the Galton Board. We used its responses primarily as suggestions rather than directly copying generated code into the project, and implemented most of the code ourselves. The project ideas and design decisions were our own; AI helped us explore implementation approaches and understand technical details. We evaluated suggestions against our code and the hardware tests described in Section 2.5.

AI also assisted with drafting and revising report text and creating some diagrams from our descriptions of the system. We reviewed and edited these materials to reflect our im-

plementation and observations. Table 3 summarizes the prompts and assistance received. Suggestions listed in the log were not necessarily implemented.

Table 3: AI prompt log

Topic	**Prompts / requests**	**How AI was used**
Histogram	"How would we generate a histogram?" "Do we define each bin first?"	Explained bin boundaries, counting landings, and normalizing bar heights.
Memory measurement	"How do I measure RAM usage?"	Provided `arm-none-eabi-size` and `nm` commands and interpreted allocation sizes.
Multicore execution	"Explain how core distribution works."	Explained the 40%/60% physics split, job queues, synchronization, and core 0's drawing responsibilities.
Drawing and physics overlap	"What is the best way to distribute work between the two cores"	Suggested arranging core 0's drawing while core 1 continues updating its ball range.
Overclocking	"Explain overclocking in terms of changing the voltage setting"	Discussed clock settings, preserving VGA timing, and testing performance on hardware.
Memory comparison	"Give me a table for this."	Summarized the reported measured static RAM reduction from **514,788 to 283,364 bytes** after the monochrome changes.
Background DMA	"What would the third DMA channel do?"	Explained cached backgrounds, flash versus RAM storage, DMA chaining, and timing tradeoffs.
CMake error debugging	"Debug this error. Does CMake need changes before building?"	Suggested compiling the VGA source, generating PIO headers, linking required libraries, and optionally enabling USB serial.
Random trajectories	"The balls are following fixed paths." "Does this only generate 101 values?"	Identified limited spawn-velocity choices and quantization as sources of repeated starting states.

2.7 Optimization Progression

Figure 7 summarizes the increase in ball capacity throughout development. Starting from approximately 550 balls, we reached 1,500 through local peg lookup, 3,000 by using both cores, and 5,600 after replacing the square-root calculation. Initial overclocking increased capacity to 7,500 balls at 200 MHz, 9,700 at 250 MHz, and 11,000 at 280 MHz.

Reducing drawing work brought capacity to approximately 13,000 balls, at which point

memory became the limiting factor. Monochrome framebuffers freed enough memory to reach approximately 14,700 balls. Further overclocking to 380 MHz brought capacity to approximately 19,000, and the subsequent scheduling and physics improvements brought the final reported capacity to 22,200 balls, which is approximately forty times the original capacity.

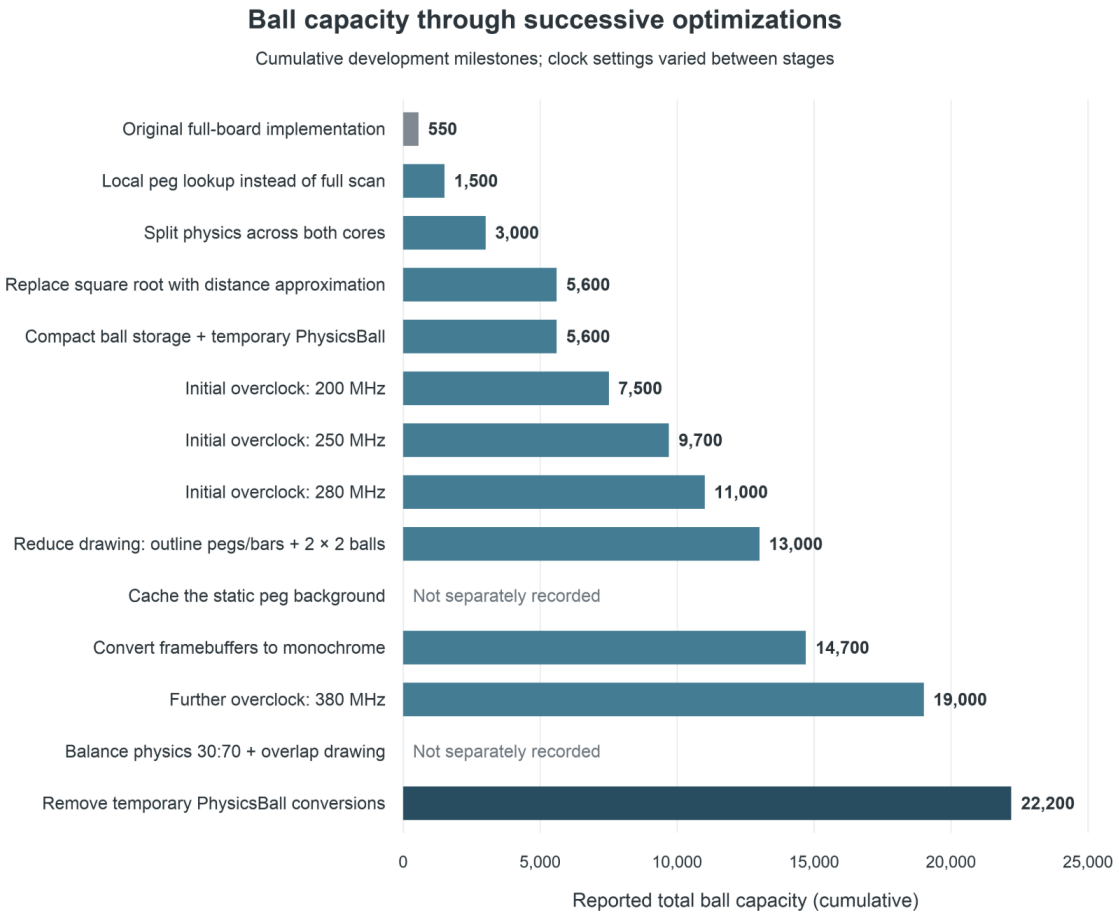


Figure 7: Cumulative ball capacity throughout optimization.

Each milestone includes the preceding optimizations, so the graph does not isolate the contribution of each change. Compact ball storage, background caching, and workload balancing are included to show the development sequence, but their individual capacity limits were not separately recorded. Clock settings also changed during development, so these results represent the progression of the complete implementation rather than comparisons under identical conditions.

2.8 Memory and Timing

The largest memory savings came from compact ball storage and the monochrome display. Each ball decreased from 16 to 12 bytes, saving 25% of the ball array. Each framebuffer de-

creased from 153,600 to 38,400 bytes, saving 75%. Together, the two framebuffer reductions freed 230,400 bytes for additional balls and other program data.

Table 4: Memory changes at fixed ball counts.

Allocation	**Before (bytes)**	**After (bytes)**
One ball	16	12
5,600-ball array	89,600	67,200
Data+bss at 5,600 balls	408,944	386,552
One framebuffer	153,600	38,400
Data+bss at 13,000 balls	514,788	283,364

At the monochrome stage, total static allocation fell by 231,424 bytes. The framebuffer changes accounted for 230,400 bytes of that reduction, with another 1,024 bytes coming from other allocations. At the final reported capacity of 22,200 balls, the ball array alone occupied

$$22{,}200 \times 12 = 266{,}400 \text{ bytes.}$$

Adding two monochrome framebuffers, the cached background, and the audio samples gives a subtotal of 386,010 bytes. This excludes other globals, stacks, and any dynamic allocations; a final compiled total was not included in our notes.

The reported capacity grew from approximately 550 to 22,200 balls through the combined optimizations. Our timing method included the background copy using `memcpy()`, physics calculations, synchronization between cores, and rendering. However, we did not retain final frame-time samples, a recorded deadline-miss count, or a sustained-run duration. The maximum ball count is therefore a reported performance milestone at the 60 fps target rather than a verified record of sustained timing behavior.

2.9 Display and Demonstration Evidence

Figure 8 shows our near-final Galton Board running with 21,300 active particles. The VGA display shows all 16 rows of pegs, the histogram forming a roughly bell-shaped distribution, and the simulation statistics in the upper-left corner. The displayed settings were a bounciness of 0.65 and gravity of 0.37.

The photograph also shows the effects of our earlier drawing optimizations. The pegs and histogram bars appear as hollow outlines rather than filled shapes, and the balls are represented by small squares instead of circles. Even with thousands of particles on the screen, the peg arrangement and histogram remain visible, although some areas appear crowded due to the large number of balls.

This photograph captures 21,300 active particles, while the maximum capacity we eventually achieved was 22,200 balls.

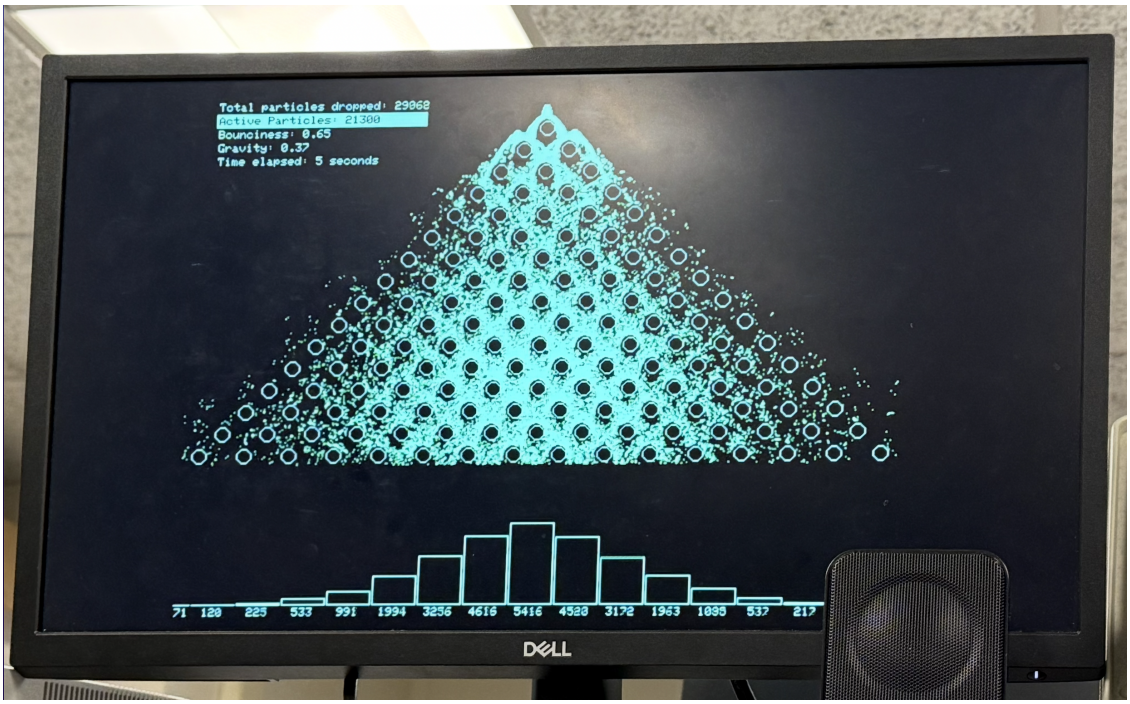


Figure 8: Near-final Galton Board demonstration with 21,300 active particles.

2.10 Distribution and Numerical Limitations

Figure 8 shows the near-final implementation with 21,300 active balls, a bounciness setting of 0.65, and gravity of 0.37. At the time of the photograph, the display reported 29,062 completed landings. The histogram has a clear central peak and decreases toward both edges, giving it an approximately bell-shaped appearance. The central bin contains 5,416 landings, while its immediate neighbors contain 4,616 and 4,528. This near symmetry is also visible farther from the center, although the corresponding bin counts are not identical.

The photograph was taken approximately five seconds after boot, so it captures an early stage of histogram accumulation. Allowing the simulation to run longer would produce more completed landings, reducing relative sampling fluctuations and making the histogram smoother. If the underlying landing distribution is approximately Gaussian, its bell-shaped form should become clearer as more samples accumulate. However, additional runtime would not remove systematic differences caused by the collision physics or numerical approximations.

These observations are qualitatively consistent with the Gaussian approximation expected from a Galton Board. However, our simulation does not implement an independent, equally likely left-or-right decision at every row. Each collision depends on the ball's incoming position and velocity, which carry information from previous collisions. Balls can also bounce upward or pass between pegs without colliding in every row. Consequently, a bell-shaped histogram does not establish exact agreement with the ideal 16-row binomial distribution. Some outer-bin counts are obscured in the photograph, so we do not use this image to calculate a complete empirical mean or standard deviation.

Our optimizations also introduced numerical limitations. The distance approximation

produces a collision normal that is not exactly unit length, affecting both velocity reflection and the displacement used to separate overlapping balls and pegs. The compact velocity representation introduces rounding into repeated updates, while discrete time steps can allow a sufficiently fast ball to cross a peg between collision checks. These effects can alter individual trajectories and potentially influence the histogram. We did not measure their contributions separately, so the photograph supports a qualitative assessment of the distribution rather than a numerical error bound or a verified Gaussian fit.

3 Conclusion

We developed an interactive digital Galton Board that combined collision physics, VGA animation, DMA-driven audio, and rotary encoder controls. Starting with a single ball and peg allowed us to establish the basic behavior before expanding to 16 rows, 136 pegs, and a landing histogram. The controls let us vary ball count, gravity, and bounciness while observing their effects on the motion and distribution. The resulting histogram showed an approximately bell-shaped distribution, although our observations were qualitative rather than a numerical verification of a Gaussian fit.

Our reported capacity increased from approximately 550 to 22,200 balls through successive changes to collision processing, rendering, memory use, and scheduling. Local peg lookup and the distance approximation reduced the work required per ball. Using both cores and adjusting their workload split allowed core 0 to draw its updated balls while core 1 continued calculating physics. Caching the static background also avoided redrawing every peg individually. Compact ball storage and monochrome framebuffers addressed memory constraints, while overclocking provided more processing capacity within each frame. These changes showed us that improving one part of the program often exposed a different limitation: after reducing drawing work, for example, memory became the obstacle to increasing the ball count.

The improvements involved tradeoffs. Smaller monochrome balls reduced visual detail, approximate collision normals and compact velocities reduced numerical precision, and busy audio playback caused some collision requests to be skipped. Although the final system reached the reported maximum ball count, we did not record a complete set of final frame times or a sustained-run duration. Future testing should measure deadline misses over longer runs and compare complete histogram data with the ideal distribution. We would also record each optimization's clock, voltage, memory use, and timing results as it was tested, making the contribution of each change easier to evaluate. Overall, the project taught us to consider computation, memory, and hardware concurrency together when improving a real-time system.

4 Answers to the Lab Questions

4.1 CLT Assumptions in the Physical Model

The classical independent and identically distributed form of the Central Limit Theorem (CLT) applies to random variables with finite mean and variance. In an ideal Galton Board,

each time a ball hits a peg, it has an equal 50% chance of bouncing left or right, regardless of what happened before. However, as Professor Hunter Adams mentions in the assigned reading, a physical Galton Board does not completely follow these assumptions.

First, the collisions are not independent because one bounce affects the next. For example, if a ball hits the right side of a peg and bounces to the right, it now has a rightward velocity. This affects the angle at which it approaches the next peg and which direction it bounces afterward. Therefore, the next collision depends partly on the previous one.

Second, the collisions are not identically distributed because every collision does not have the same chance of sending the ball left or right. For example, a ball hitting a peg almost directly in the center may bounce differently from one that barely touches its edge. The ball's incoming speed and angle also affect its direction after the collision. Unlike the ideal model, our code calculates these bounces using collision physics instead of randomly choosing left or right with equal probability.

The finite variance assumption is reasonable for our model because the board has a limited width and the ball cannot land infinitely far to the left or right. This means the possible horizontal landing positions are bounded, so their variance is finite.

In our simulation, we added randomness by giving each ball a small random horizontal velocity when it spawned. After that, its movement followed our gravity and collision calculations rather than making a new random decision at every peg.

Even though the first two assumptions do not hold, our histogram still formed a roughly bell-shaped distribution. Professor Hunter explained this using a generalization of the CLT for mixing processes. The idea is that even when consecutive events depend on each other, the influence of an earlier event can become weaker over time. For example, a ball might bounce strongly to the right after hitting one peg, but after hitting several more pegs, that first bounce may have much less influence on its final direction.

This helps explain why our Galton Board can produce a Gaussian-like distribution even without perfectly independent or identical collisions. However, our histogram alone does not prove that the mathematical conditions for this generalized CLT are satisfied.

4.2 Why More Rows Improve the Approximation

In an ideal Galton Board, each row contributes an independent left-or-right decision with equal probability. The number of rightward decisions after n rows therefore follows a binomial distribution with parameters n and $p = 0.5$. As the number of rows increases, the central limit theorem implies that this sum, after centering and scaling, approaches a standard Gaussian distribution. The bin spacing also becomes smaller relative to the distribution's overall width, making the discrete histogram more closely resemble a continuous bell-shaped curve.

Increasing the number of rows improves the theoretical Gaussian approximation, whereas dropping more balls through a fixed board reduces sampling noise. More balls make the measured histogram better represent the board's underlying distribution, but do not change that distribution into a Gaussian.

4.3 Ideal 16-Row Mean and Standard Deviation

For the ideal board, we number the landing bins from 0 to 16 and let X be the number of rightward decisions a ball makes across the 16 rows. If each decision is independent and equally likely to go left or right, then

$$X \sim \text{Binomial}(16, 0.5).$$

The number of rightward decisions determines the landing bin. Using the binomial mean and variance gives

$$\mu = np = 16(0.5) = 8, \qquad \sigma^2 = np(1-p) = 16(0.5)(0.5) = 4.$$

The expected landing position is therefore bin 8, with a standard deviation of $\sigma = 2$ bins. The Gaussian approximation is $\mathcal{N}(8, 4)$, where the second parameter is the variance.

If we measure position from the center bin, the mean is zero and the standard deviation is still two bins. For bin spacing d, the horizontal displacement is $Z = d(X - 8)$, so its standard deviation is $2d$. These values describe the ideal independent-decision model. Our collision simulation carries position and velocity from one row to the next, so its measured distribution may differ from this model.

A Code Appendix

A.1 Main Program: `galton_board.c`

Listing 1: Main Galton Board program.

```
/**
 * hunter adams (vha3@cornell.edu)
 *
 * galton board with bouncing balls and a landing histogram
 * serial input sets a color variable, though drawing currently uses white
 *
 * hardware connections
    - gpio 14 ---> rotary encoder b
    - gpio 15 ---> rotary encoder a
    - gpio 16 ---> vga hsync
    - gpio 17 ---> vga vsync
    - gpio 18 ---> vga green lo-bit --> 470 ohm resistor --> vga_green
    - gpio 19 ---> vga green hi_bit --> 330 ohm resistor --> vga_green
    - gpio 20 ---> 330 ohm resistor ---> vga-blue
    - gpio 21 ---> 330 ohm resistor ---> vga-red
    - gpio 22 ---> pull up ---> pin 1 of rotory encoder
    - gpio 26 ---> gpio 22 ---> detects pull low


    - rp2040 gnd ---> vga-gnd
    - rp2040 gnd ---> rotary encoder com
 *
 * resources used
 *  - pio state machines 0, 1, and 2 on pio instance 0
 *  - dma channels (2, by claim mechanism)
 *  - 153.6 kbytes of ram (for pixel color data)
 *
 */

// include the vga grahics library
#include "VGA/vga16_graphics_v3.h"
// include standard libraries
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdbool.h>
// include pico libraries
#include "pico/stdlib.h"
#include "pico/divider.h"
#include "pico/multicore.h"
#include "pico/sync.h"
#include "pico/util/queue.h"
// include hardware libraries
#include "hardware/pio.h"
#include "hardware/spi.h"
#include "hardware/dma.h"
#include "hardware/clocks.h"
#include "hardware/pll.h"
#include "hardware/sync.h"
// include protothreads
#include "pt_cornell_rp2040_v1_4.h"

#include "hardware/vreg.h"

// fixed-point values scaled by 32768, with 15 fractional bits
typedef signed int fix15;

```

```
61  #define multfix15(a,b) \
62      ((fix15)((((signed long long)(a)) * ((signed long long)(b))) >> 15))
63
64  #define float2fix15(a) ((fix15)((a) * 32768.0))
65  #define fix2float15(a) ((float)(a) / 32768.0f)
66  #define absfix15(a)    (abs((a)))
67
68  #define int2fix15(a) \
69      ((fix15)(((signed long long)(a)) * 32768LL))
70
71  #define fix2int15(a) ((int)((a) >> 15))
72  #define char2fix15(a) int2fix15(a)
73
74  // the caller must ensure b != 0
75  #define divfix(a,b) \
76      ((fix15)div_s64s64(((signed long long)(a)) * 32768LL, \
77                         ((signed long long)(b))))
78
79  // board dimensions and limits, matched to the vga driver
80  #define SCREEN_WIDTH 640
81  #define SCREEN_HEIGHT 480
82  #define SIDE_MARGIN 20
83  #define LEFT_WALL SIDE_MARGIN
84  #define RIGHT_WALL (SCREEN_WIDTH - SIDE_MARGIN)
85  #define TOP_WALL 0
86  #define BOARD_CENTER_X ((LEFT_WALL + RIGHT_WALL) / 2)
87  #define FIRST_PEG_Y 45
88  #define peg_radius 6
89  #define ball_radius 2
90  #define COLLISION_CLEARANCE 1
91  #define COLLISION_RADIUS (peg_radius + ball_radius)
92  #define vertical_seperation 19
93  #define horizonal_seperation 38
94  #define SPAWN_MAX_VX 0.5f
95  // set to 0 to compare with single-core physics
96  #define USE_SECOND_CORE 1
97  // ball capacity is independent of peg/bin storage
98  #define max_array 22200
99  #define MAX_PEG_ROWS 16
100 #define MAX_PEGS (MAX_PEG_ROWS * (MAX_PEG_ROWS + 1) / 2)
101 #define MAX_BINS (MAX_PEG_ROWS + 1)
102 #define HIST_BOTTOM_MARGIN 10
103 #define HIST_MIN_HEIGHT (2 * vertical_seperation)
104 #define TEXT_MARGIN 20
105 #define TEXT_LINE_SPACING 12
106 #define MAX_COLOR 15
107
108 // top and side checks allow for ball radius, bottom check marks a landing
109 #define hitBottom(b) ((b) >= int2fix15(board_bottom_y))
110 #define hitTop(b) ((b) < int2fix15(TOP_WALL + ball_radius))
111 #define hitLeft(a) ((a) < int2fix15(LEFT_WALL + ball_radius))
112 #define hitRight(a) ((a) > int2fix15(RIGHT_WALL - ball_radius))
113
114 // scope probes: use only if gpio 2 and 3 are free on your board
115 // gpio 2 high: all frame work gpio 3 high: combined physics and ball drawing
116 #define FRAME_PROBE 2
117 #define PHYSICS_PROBE 3
118
119 // driver setup expects double buffering on rp2350
120 // buffer-swap signal controls when each frame starts
121 // clock speed and button polling interval
122 #define SYSTEM_CLOCK_KHZ 380000
123 #define BUTTON_POLL_US 10000
124
125 // rotary encoder
126 #define A_GPIO 15
127 #define B_GPIO 14
128
```

```
#define PULL_UP_GPIO 22
#define DETECT_PULL_UP 26


// dac
#define DAC_CS_GPIO   5
#define DAC_SCK_GPIO  6
#define DAC_MOSI_GPIO 7
#define DAC_SPI       spi0

#define AUDIO_SAMPLE_RATE 22050
#define SOUND_DURATION_MS 100
#define SOUND_SAMPLES (AUDIO_SAMPLE_RATE * SOUND_DURATION_MS / 1000)
#define TONE_FREQUENCY_HZ 400.0f
#define TONE_AMPLITUDE 1500.0f
#define DAC_BITS 12
#define DAC_MIDPOINT (1u << (DAC_BITS - 1))
#define DAC_SAMPLE_MASK ((1u << DAC_BITS) - 1u)
#define DAC_SPI_BAUD 20000000
#define DAC_CONFIG_A      0x3000     // dac control bits selecting a, 1x gain, active output

// 60fps in microseconds
#define FRAME_RATE_US (1000000/60)

// the led is connected to gpio 25
#define LED_PIN 25

static uint16_t sound_data[SOUND_SAMPLES]; // sequence of 16-bit words dma will send to dac

// dma
static int data_chan;
static int ctrl_chan;

// control channel copies this pointer into the data channel
static uint16_t *address_pointer = sound_data;

// storage capacity is separate from the number of active bins
fix15 bin_edges[MAX_BINS + 1];
uint32_t bins[MAX_BINS] = {0};
// independent of histogram rescaling: the exact landings since its last reset
uint64_t balls_fallen = 0;
int num_bins = 0;
int board_bottom_y;
int histogram_top_y;
int histogram_height;

// serial color setting, currently unused by ball drawing
// balls are drawn in white in the animation thread
char color = PINK ;

// velocities and collision normals use signed q8.8 (16 bits total)
// positions retain q15 so the full 640 x 480 screen fits
#define Q8_ONE 256
#define float2q8(value) ((int16_t)((value) * 256.0f))
#define q8ToPosition(value) ((fix15)(value) * 128)

// multiply with extra room, then clamp to the signed 16-bit range
static inline int16_t multQ8(int16_t a, int16_t b)
{
    int32_t result = ((int32_t)a * b) / Q8_ONE;
    if (result > INT16_MAX) result = INT16_MAX;
    if (result < INT16_MIN) result = INT16_MIN;
    return (int16_t)result;
}

// unused position and velocity variables
fix15 boid0_x ;
fix15 boid0_y ;
```

```
fix15 boid0_vx ;
fix15 boid0_vy ;

int16_t gravity = float2q8(0.37f);
int16_t bounciness = float2q8(0.65f);

// unused second set of position and velocity variables
fix15 boid1_x ;
fix15 boid1_y ;
fix15 boid1_vx ;
fix15 boid1_vy ;

// peg positions in fixed-point coordinates

struct Peg{
    fix15 x;
    fix15 y;
};

struct Peg pegs[MAX_PEGS];

int num_peg_rows=16;
int total_peg = 0;

struct Ball {
    fix15 x;
    fix15 y;
    int16_t vx;
    int16_t vy;
};

// active ball count and storage
int num_balls = 0;
struct Ball balls[max_array];

// encoder count and previous channel readings
int rotary_encoder_num;
static volatile int encoder_delta = 0;
bool prevA;
bool prevB;
bool A;
bool B;

int detect;
int prev_detect=1;
int rainbow=0;

int parameter_selected=0;

// core 0 changes parameters/counts only between physics jobs
// each worker owns a disjoint range of balls for the whole job
struct PhysicsJob { int first; int end; };
static queue_t physics_jobs;
static queue_t physics_done;
// mutex reserved for spawning, but not locked by the current code
// both physics cores currently call rand without this lock
static mutex_t spawn_random_mutex;
// each core owns its landing counters during a job
static uint32_t frame_landings[2][MAX_BINS];
// each core writes its own slot; core 0 reads only after the join
static bool collision_sound_requested[2];

static inline int16_t randomSpawnVelocity(void)
{
    // 257 possible sideways speeds, from -0.5 to +0.5 pixels per frame
    return (int16_t)(rand() % 257 - 128);
}

```

```
265 static inline fix15 randomSpawnOffset(void)
266 {
267     // small random sideways offset to vary the starting path
268     return float2fix15((rand() % 10001 - 5000) / 10000.0f);
269 }
270
271 // start above the first peg with a small random offset and sideways speed
272 void resetBall(struct Ball *ball)
273 {
274     int spawn_gap = vertical_seperation;
275     if (spawn_gap < COLLISION_RADIUS + COLLISION_CLEARANCE)
276         spawn_gap = COLLISION_RADIUS + COLLISION_CLEARANCE;
277     fix15 spawn_y = pegs[0].y - int2fix15(spawn_gap);
278     if (spawn_y < int2fix15(TOP_WALL + ball_radius))
279         spawn_y = int2fix15(TOP_WALL + ball_radius);
280
281     *ball = (struct Ball){
282         .x = pegs[0].x + randomSpawnOffset(),
283         .y = spawn_y,
284         .vx = randomSpawnVelocity(),
285         .vy = 0
286     };
287 }
288
289 // add one ball if there is space and at least one peg
290 void spawnBoid(void)
291 {
292     if (num_balls >= max_array || total_peg == 0) return;
293     resetBall(&balls[num_balls++]);
294 }
295
296 void resetHistogram(void)
297 {
298     // clear landing counts between physics jobs on core 0
299     memset(bins, 0, sizeof(bins));
300     memset(frame_landings, 0, sizeof(frame_landings));
301     balls_fallen = 0;
302 }
303
304 // match ball count to the encoder setting, reset counts when it changes
305 void syncBallCount(void)
306 {
307     if (rotary_encoder_num < 0) rotary_encoder_num = 0;
308     if (rotary_encoder_num > max_array) rotary_encoder_num = max_array;
309     int target = rotary_encoder_num;
310
311     if (total_peg == 0) {
312         num_balls = 0;
313         return;
314     }
315     if (num_balls == target) return;
316
317     // keep remaining balls moving from their current positions
318     resetHistogram();
319     if (num_balls > target) num_balls = target;
320     while (num_balls < target) spawnBoid();
321 }
322
323 // bottom-row peg centers divide the board into num_peg_rows + 1 bins
324 void init_bins(void) {
325     int bottom_row_start = total_peg - num_peg_rows;
326     num_bins = num_peg_rows + 1;
327     resetHistogram();
328     bin_edges[0] = int2fix15(LEFT_WALL);
329
330     for (int i = 0; i < num_peg_rows; i++) {
331         bin_edges[i + 1] = pegs[bottom_row_start + i].x;
332     }
```

```
333
334     bin_edges[num_bins] = int2fix15(RIGHT_WALL);
335
336     // landing line below the last pegs, with room for the ball radius
337     board_bottom_y = fix2int15(pegs[bottom_row_start].y)
338                    + COLLISION_RADIUS + COLLISION_CLEARANCE;
339     histogram_top_y = board_bottom_y+50;
340     histogram_height = SCREEN_HEIGHT - HIST_BOTTOM_MARGIN
341                      - histogram_top_y - 1;
342 }
343
344 void recordLanding(fix15 x)
345 {
346     // binary search for the landing bin, boundaries go to the right bin
347     // positions outside the edges go into the first or last bin
348     int low = 0;
349     int high = num_bins - 1;
350     while (low < high) {
351         int mid = low + (high - low) / 2;
352         if (x < bin_edges[mid + 1]) high = mid;
353         else low = mid + 1;
354     }
355
356     // record in this core's own counters
357     frame_landings[get_core_num()][low]++;
358 }
359
360 static void mergeFrameLandings(void)
361 {
362     // combine both cores' counts after physics finishes
363     // 64-bit sums avoid overflow before rescaling
364     uint64_t updated_bins[MAX_BINS];
365     uint32_t landed = 0;
366     bool rescale = false;
367     for (int i = 0; i < num_bins; i++) {
368         uint32_t added = frame_landings[0][i] + frame_landings[1][i];
369         landed += added;
370         updated_bins[i] = (uint64_t)bins[i] + added;
371         if (updated_bins[i] > UINT32_MAX) rescale = true;
372         frame_landings[0][i] = 0;
373         frame_landings[1][i] = 0;
374     }
375     // halve all bins if any count is too large
376     // keep the exact total since reset seperate from scaled bin counts
377     for (int i = 0; i < num_bins; i++) {
378         bins[i] = (uint32_t)(rescale ? updated_bins[i] / 2 : updated_bins[i]);
379     }
380     balls_fallen += landed;
381 }
382
383 // read both encoder channels and collect signed rotation steps
384 static void encoder_callback(uint gpio, uint32_t events)
385 {
386     if (gpio != A_GPIO && gpio != B_GPIO) return;
387     (void)events;
388     uint32_t pins = gpio_get_all();
389     A = (pins & (1u << A_GPIO)) == 0;
390     B = (pins & (1u << B_GPIO)) == 0;
391     int delta = (A ^ prevB) - (prevA ^ B);
392     prevA = A;
393     prevB = B;
394     // limit pending encoder steps to keep later maths in range
395     if ((delta > 0 && encoder_delta < max_array) ||
396         (delta < 0 && encoder_delta > -max_array)) {
397         encoder_delta += delta;
398     }
399 }
400
```

```
401 // apply pending rotation to the selected setting before physics starts
402 static void applyEncoderChanges(void)
403 {
404     if (encoder_delta == 0) return;
405 
406     uint32_t irq_state = save_and_disable_interrupts();
407     int delta = encoder_delta;
408     encoder_delta = 0;
409     restore_interrupts(irq_state);
410 
411     if (parameter_selected==0){
412         rotary_encoder_num += delta;
413         if (rotary_encoder_num < 0) rotary_encoder_num = 0;
414         if (rotary_encoder_num > max_array) rotary_encoder_num = max_array;
415     }
416     if (delta!=0) resetHistogram();
417     if (parameter_selected == 1 || parameter_selected == 2) {
418         int16_t *parameter = parameter_selected == 1 ? &bounciness : &gravity;
419         // limit the step size before adjusting gravity or bounciness
420         if (delta > Q8_ONE) delta = Q8_ONE;
421         if (delta < -Q8_ONE) delta = -Q8_ONE;
422         int16_t updated = (int16_t)(*parameter + delta * float2q8(0.01f));
423         if (updated < 0) updated = 0;
424         if (updated > Q8_ONE) updated = Q8_ONE;
425         *parameter = updated;
426     }
427 }
428 
429 // check the button and cycle through the three settings
430 static PT_THREAD (poll_encoder_button(struct pt *pt))
431 {
432     PT_BEGIN(pt);
433     static int button_candidate = 1;
434     static int button_samples = 0;
435     while (1) {
436         detect = gpio_get(DETECT_PULL_UP);
437 
438         // wait for three matching readings to filter button bounce
439         if (detect != button_candidate) {
440             button_candidate = detect;
441             button_samples = 1;
442         } else if (button_samples < 3) {
443             button_samples++;
444         }
445 
446         if (button_samples == 3 && button_candidate != prev_detect) {
447             prev_detect = button_candidate;
448 
449             // advance the selection once per stable button press
450             if (prev_detect == 0) {
451                 parameter_selected = (parameter_selected + 1) % 3;
452             }
453         }
454         PT_YIELD_usec(BUTTON_POLL_US);
455     }
456     PT_END(pt);
457 }
458 
459 
460 void playCollisionSound(void)
461 {
462     // start control channel -> reset buffer address ->
463     // trigger data channel -> send sound -> stop
464 
465     // let an existing sound finish
466     if (dma_channel_is_busy(ctrl_chan) ||
467         dma_channel_is_busy(data_chan) ||
468         spi_is_busy(DAC_SPI)) {
```

```
        return;
    }

    // prepare both transfer counts without starting either channel
    dma_channel_set_trans_count(data_chan, SOUND_SAMPLES, false);
    dma_channel_set_trans_count(ctrl_chan, 1, false);

    // rewind the sample pointer, then start playback through chaining
    dma_start_channel_mask(1u << ctrl_chan);
}


// estimate distance as the larger offset plus half the smaller one
static fix15 alphaMaxBetaMin(fix15 dx, fix15 dy)
{
    fix15 ax = absfix15(dx);
    fix15 ay = absfix15(dy);

    fix15 maximum = ax > ay ? ax : ay;
    fix15 minimum = ax > ay ? ay : ax;

    return maximum + (minimum >> 1);
}

// resolve peg overlap, bounce approaching balls, and request a sound
void updateBallPosition(struct Ball *ball,
                        struct Peg *peg)
{
    // calculate displacement from peg to ball
    fix15 dx = ball->x - peg->x;
    fix15 dy = ball->y - peg->y;

    // quick box check, then squared-distance check for overlap
    if (absfix15(dx) < int2fix15(COLLISION_RADIUS) && absfix15(dy) < int2fix15(
        COLLISION_RADIUS)) {
        fix15 distance_squared = multfix15(dx, dx) + multfix15(dy, dy);
        const fix15 radius = int2fix15(COLLISION_RADIUS);
        if (distance_squared < multfix15(radius, radius)) {
            fix15 distance = alphaMaxBetaMin(dx, dy);
            // estimate the outward collision direction
            int16_t normal_x;
            int16_t normal_y;
            if (distance > 0) {
                // convert the direction to an 8-fractional-bit value
                normal_x = (int16_t)(((int32_t)dx * Q8_ONE) / distance);
                normal_y = (int16_t)(((int32_t)dy * Q8_ONE) / distance);
            } else { // matching centers, use upward as the fallback direction
                normal_x = 0;
                normal_y = -Q8_ONE;
            }

            // motion along normal direction
            int16_t dot_product = (int16_t)(multQ8(normal_x, ball->vx)
                                + multQ8(normal_y, ball->vy));

            // move outward using the collision radius and a small gap
            ball->x = peg->x + q8ToPosition(normal_x) * (COLLISION_RADIUS +
                COLLISION_CLEARANCE);
            ball->y = peg->y + q8ToPosition(normal_y) * (COLLISION_RADIUS +
                COLLISION_CLEARANCE);

            // if ball is approaching
            if (dot_product < 0) {
                int16_t impulse = (int16_t)(-2 * dot_product);

                // reflect velocity
                ball->vx += multQ8(normal_x, impulse);
                ball->vy += multQ8(normal_y, impulse);
```

```
534
535                 // scale both velocity components by bounciness
536                 ball->vx = multQ8(ball->vx, bounciness);
537                 ball->vy = multQ8(ball->vy, bounciness);
538
539                 collision_sound_requested[get_core_num()] = true;
540             }
541         }
542     }
543 }
544
545 // draw scaled landing bars with counts underneath
546 void drawHist(void)
547 {
548     if (num_bins == 0) return;
549     // find the tallest bin and room needed for count labels
550     // bars scale to fit above the labels
551     int label_rows = 1;
552
553     uint32_t peak = 0;
554     for (int i = 0; i < num_bins; i++) {
555         if (bins[i] > peak) peak = bins[i];
556         char count_text[11]; // up to 10 digits plus the string terminator
557         snprintf(count_text, sizeof(count_text), "%" PRIu32, bins[i]);
558         int width = fix2int15(bin_edges[i + 1]) - fix2int15(bin_edges[i]);
559         int chars_per_row = (width - 1) / 6;
560         if (chars_per_row < 1) chars_per_row = 1;
561         int rows = ((int)strlen(count_text) + chars_per_row - 1) / chars_per_row;
562         if (rows > label_rows) label_rows = rows;
563     }
564     // reserve space for count labels using the 6 by 8 pixel font
565     // wrap long counts inside each bin
566     int bar_height = histogram_height - label_rows * 8 - 2;
567     if (bar_height <= 0) return;
568     int baseline_y = histogram_top_y + bar_height;
569     setTextColor(WHITE);
570     setTextSize(1);
571
572     for (int i = 0; i < num_bins; i++) {
573         int x = fix2int15(bin_edges[i]);
574         int width = fix2int15(bin_edges[i + 1]) - x;
575         // leave one pixel between bars so neighboring bins stay visible
576         if (width > 1) width--;
577         if (width <= 0) continue;
578
579         // scale bar heights relative to the largest count
580         if (peak > 0 && bins[i] > 0) {
581             uint64_t height = (uint64_t)bins[i] * bar_height / peak;
582             if (height == 0) height = 1;
583             drawRect(x, baseline_y - (int)height, width, (int)height,  WHITE);
584         }
585
586         char count_text[11];
587         snprintf(count_text, sizeof(count_text), "%" PRIu32, bins[i]);
588         int chars_per_row = width / 6;
589         if (chars_per_row < 1) chars_per_row = 1;
590         int length = (int)strlen(count_text);
591         for (int offset = 0, row = 0; offset < length; row++) {
592             int chars = length - offset;
593             if (chars > chars_per_row) chars = chars_per_row;
594             char line[11];
595             memcpy(line, count_text + offset, chars);
596             line[chars] = '\0';
597             drawTextGLCD(x + (width - chars * 6) / 2,
598                          baseline_y + 2 + row * 8, line,  WHITE, BLACK);
599             offset += chars;
600         }
601     }
```

```
602 }
603
604
605 void generate_pegs(void)
606 {
607     // limit peg rows to fit the screen and storage
608     int max_rows_width = (RIGHT_WALL - LEFT_WALL
609                         - 2 * (COLLISION_RADIUS + COLLISION_CLEARANCE))
610                         / horizonal_seperation + 1;
611     int max_rows_height = (SCREEN_HEIGHT - HIST_BOTTOM_MARGIN - 1
612                          - HIST_MIN_HEIGHT - FIRST_PEG_Y
613                          - COLLISION_RADIUS - COLLISION_CLEARANCE)
614                          / vertical_seperation + 1;
615     int max_rows = max_rows_width;
616     if (max_rows_height < max_rows) max_rows = max_rows_height;
617     if (max_rows > MAX_PEG_ROWS) max_rows = MAX_PEG_ROWS;
618     while (max_rows > 0 && (long long)max_rows * (max_rows + 1) / 2 > MAX_PEGS)
619         max_rows--;
620
621     // stop if there is no room for a full row
622     if (max_rows < 1) {
623         total_peg = 0;
624         num_bins = 0;
625         return;
626     }
627     if (num_peg_rows < 1) num_peg_rows = 1;
628     if (num_peg_rows > max_rows) num_peg_rows = max_rows;
629
630     total_peg = 0;
631     for (int row = 1; row <= num_peg_rows; row++) {
632         // center each row, adding one more peg per row
633         fix15 starting_x = int2fix15(BOARD_CENTER_X)
634                          - int2fix15((row - 1) * horizonal_seperation) / 2;
635         fix15 y = int2fix15(FIRST_PEG_Y + (row - 1) * vertical_seperation);
636         for (int col = 0; col < row; col++) {
637             struct Peg newPeg = {
638                 starting_x + int2fix15(horizonal_seperation * col), y
639             };
640             pegs[total_peg++] = newPeg;
641         }
642     }
643     init_bins();
644 }
645
646 // use the regular peg spacing to check only the nearest candidate
647 void checkNearbyPeg(struct Ball *ball)
648 {
649     if (total_peg == 0) return;
650
651     const fix15 row_spacing = int2fix15(vertical_seperation);
652     const fix15 col_spacing = int2fix15(horizonal_seperation);
653     const fix15 radius = int2fix15(COLLISION_RADIUS);
654
655     // find the closest peg row from the ball height
656     int row = (ball->y - pegs[0].y + row_spacing / 2)
657             / row_spacing;
658
659     if (row < 0 || row >= num_peg_rows) return;
660
661     // each row has one more peg than the previous row
662     // triangular sum gives the first peg index in this row
663     int first = row * (row + 1) / 2;
664
665     // reject balls too far above or below this row
666     if (absfix15(ball->y - pegs[first].y) >= radius) return;
667
668     // find the nearest column relative to this row's first peg
669     int col = (ball->x - pegs[first].x + col_spacing / 2)
```

```
            / col_spacing;

    // zero-based row r contains r + 1 pegs
    if (col < 0 || col > row) return;

    struct Peg *peg = &pegs[first + col];

    if (absfix15(ball->x - peg->x) >= radius) return;

    // check overlap with the nearest peg and handle any bounce
    updateBallPosition(ball, peg);
}

// move one ball, handle walls and landings, then apply gravity
void updateOneBall(struct Ball *ball)
{
    // move by the current velocity, then check the nearest peg
    ball->x += q8ToPosition(ball->vx);
    ball->y += q8ToPosition(ball->vy);

    checkNearbyPeg(ball);

    if (hitTop(ball->y)) {
        if (ball->vy < 0)
            ball->vy = -ball->vy;
        ball->y = int2fix15(TOP_WALL + ball_radius);
    }

    if (hitRight(ball->x)) {
        if (ball->vx > 0)
            ball->vx = -ball->vx;
        ball->x = int2fix15(RIGHT_WALL - ball_radius);
    }

    if (hitLeft(ball->x)) {
        if (ball->vx < 0)
            ball->vx = -ball->vx;
        ball->x = int2fix15(LEFT_WALL + ball_radius);
    }

    if (hitBottom(ball->y)) {
        recordLanding(ball->x);

        // restart this ball above the first peg
        resetBall(ball);
        return;
    }

    ball->vy += gravity;

    // cap each speed component to leave room for collision maths
    const int16_t speed_limit = 24 * Q8_ONE;
    if (ball->vx > speed_limit) ball->vx = speed_limit;
    if (ball->vx < -speed_limit) ball->vx = -speed_limit;
    if (ball->vy > speed_limit) ball->vy = speed_limit;
    if (ball->vy < -speed_limit) ball->vy = -speed_limit;

}

// core 1 waits for a ball range, updates it, then signals completion
static void core1_entry(void)
{
    while (true) {
        struct PhysicsJob job;
        queue_remove_blocking(&physics_jobs, &job);
        for (int i = job.first; i < job.end; ++i) {
            updateOneBall(&balls[i]);
        }
```

```
        // completion queue makes updated positions and counts visible to core 0
        uint32_t done = 1;
        queue_add_blocking(&physics_done, &done);
    }
}

// ================================================
// === serial input for the color setting
// ================================================
static PT_THREAD (protothread_serial(struct pt *pt))
{
    PT_BEGIN(pt);

    // stores user input
    static int user_input ;

    // wait for 1 sec
    PT_YIELD_usec(1000000) ;

    // print the protothreads version
    sprintf(pt_serial_out_buffer, "Protothreads RP2040 v1.4\n\r");

    // non-blocking write
    serial_write ;

    while(1) {
        // print prompt
        sprintf(pt_serial_out_buffer, "input a number in the range 1-%d: ", MAX_COLOR);

        // non-blocking write
        serial_write ;

        // read serial input while allowing other threads to run
        serial_read ;

        // convert input string to number
        if (sscanf(pt_serial_in_buffer, "%d", &user_input) != 1) continue;

        // store a valid color choice, currently unused by rendering
        if ((user_input > 0) && (user_input <= MAX_COLOR)) {
            color = (char)user_input ;
        }
    } // end input loop

    PT_END(pt);
} // serial thread


void draw_pegs(){
    for (int i = 0; i < total_peg; i++){
        drawCircle(
            fix2int15(pegs[i].x),
            fix2int15(pegs[i].y),
            peg_radius,
            WHITE
        );
    }
}

static uint32_t last_frame_us, last_update_draw_us, missed_frames;

// show landing total, settings, and uptime with the selected setting highlighted
void drawStats(void)
{
    char stats_text[48];
    int x = LEFT_WALL + TEXT_MARGIN;
    int y = TOP_WALL + TEXT_MARGIN;
    int normal_color = WHITE;
```

```
806     uint64_t boot_seconds = time_us_64() / 1000000u;
807 
808     setTextSize(1);
809     setTextColor(normal_color);
810 
811     snprintf(stats_text, sizeof(stats_text),
812              "Total particles dropped: %" PRIu64, balls_fallen);
813     drawTextGLCD(x, y, stats_text, normal_color, BLACK);
814 
815     // 0 = particles, 1 = bounciness, 2 = gravity
816     for (int field = 0; field < 3; field++) {
817         int field_y = y + (field + 1) * TEXT_LINE_SPACING;
818         bool selected = parameter_selected == field;
819 
820         if (selected) {
821             fillRect(x - 2, field_y - 2, 180, TEXT_LINE_SPACING, WHITE);
822             setTextColor(0);
823         } else {
824             setTextColor(WHITE);
825         }
826 
827         if (field == 0) {
828             snprintf(stats_text, sizeof(stats_text), "Active Particles: %d", num_balls);
829         } else if (field == 1) {
830             snprintf(stats_text, sizeof(stats_text), "Bounciness: %.2f", ((float)bounciness
                    / Q8_ONE));
831         } else {
832             snprintf(stats_text, sizeof(stats_text), "Gravity: %.2f", ((float)gravity /
                    Q8_ONE));
833         }
834 
835         drawTextGLCD(x, field_y, stats_text, selected ? BLACK : normal_color, selected ? (
                WHITE) : BLACK);
836     }
837 
838     setTextColor(normal_color);
839     snprintf(stats_text, sizeof(stats_text),
840              "Time elapsed: %" PRIu64 " seconds", boot_seconds);
841     drawTextGLCD(x, y + 4 * TEXT_LINE_SPACING, stats_text, normal_color, BLACK);
842 
843 }
844 
845 // the cornell v3 driver exports this draw-buffer pointer
846 extern char *current_draw_buffer;
847 
848 // cached black background and white pegs, one bit per pixel
849 // copy assumes the draw buffer uses the same packed format
850 static uint8_t peg_background[SCREEN_WIDTH * SCREEN_HEIGHT / 8];
851 static int cached_peg_count = -1;
852 
853 // rebuild the peg cache when the count changes, otherwise reuse it
854 static void restorePegBackground(void)
855 {
856     uint8_t *buffer = (uint8_t *)current_draw_buffer;
857 
858     if (cached_peg_count != total_peg) {
859         clearLowFrame(0, BLACK);
860         draw_pegs();
861 
862         // save the already-packed monochrome pixels
863         memcpy(peg_background, buffer, sizeof(peg_background));
864 
865         cached_peg_count = total_peg;
866     } else {
867         // restore the backgound and pegs in one copy
868         memcpy(buffer, peg_background, sizeof(peg_background));
869     }
870 }
```

```
871
872 // static uint8_t ball_half_width[ball_radius + 1];
873
874 // static void initballshape(void)
875 // {
876 //     // match v3 fillcircle's shape exactly; the radius is still configurable
877 //     for (int dy = 0; dy <= ball_radius; ++dy) {
878 //         ball_half_width[dy] = (uint8_t)sqrtf(
879 //             (float)(ball_radius * ball_radius + ball_radius - dy * dy));
880 //     }
881 // }
882
883 // static void drawballfast(uint8_t *buffer, int x, int y, uint8_t color)
884 // {
885 //     if (x - ball_radius < 0 || x + ball_radius >= screen_width ||
886 //         y - ball_radius < 0 || y + ball_radius >= screen_height) {
887 //         fillcircle(x, y, ball_radius, color);
888 //         return;
889 //     }
890 //     uint8_t pair = color | (color << 4);
891 //     for (int dy = -ball_radius; dy <= ball_radius; ++dy) {
892 //         int half = ball_half_width[dy < 0 ? -dy : dy];
893 //         int left = x - half;
894 //         int width = 2 * half; // same pixel span as v3 fillcircle
895 //         uint8_t *p = buffer + (y + dy) * (screen_width / 2) + (left >> 1);
896 //         if ((left & 1) && width) {
897 //             *p = (*p & 0x0f) | (color << 4);
898 //             ++p;
899 //             --width;
900 //         }
901 //         while (width >= 2) {
902 //             *p++ = pair;
903 //             width -= 2;
904 //         }
905 //         if (width) *p = (*p & 0xf0) | color;
906 //     }
907 // }
908
909 // draw each ball as a small 2 by 2 pixel square
910 static void drawBall2x2(int x, int y, uint8_t color)
911 {
912     fillRect( x -1, y -1, 2, 2, color);
913 }
914
915 // draw the requested range of balls into the current draw buffer
916 static void drawBallRange(uint8_t *buffer,
917                           int first, int end,
918                           uint8_t color)
919 {
920     for (int i = first; i < end; ++i) {
921         drawBall2x2(
922                     fix2int15(balls[i].x),
923                     fix2int15(balls[i].y),
924                     color);
925     }
926 }
927
928 // split physics work between cores, draw finished ranges, and combine results
929 void updateAndDrawBalls(uint8_t *buffer, uint8_t ball_color)
930 {
931     if (total_peg == 0 || num_bins == 0 || num_balls == 0)
932         return;
933
934     collision_sound_requested[0] = false;
935     collision_sound_requested[1] = false;
936
937 #if USE_SECOND_CORE
938     int split = num_balls * 3 / 10;
```

```
939     struct PhysicsJob job = { split, num_balls };
940 
941     // core 1 updates the remaining roughly 70 percent of balls
942     queue_add_blocking(&physics_jobs, &job);
943 
944     // core 0 updates the first roughly 30 percent of balls
945     for (int i = 0; i < split; ++i) {
946         updateOneBall(&balls[i]);
947     }
948 
949     // draw core 0's finished range
950     // core 1 may still be updating its range
951     drawBallRange(buffer, 0, split, ball_color);
952 
953     // wait before reading any balls belonging to core 1
954     uint32_t done;
955     queue_remove_blocking(&physics_done, &done);
956 
957     // draw core 1's range after its updates finish
958     drawBallRange(buffer, split, num_balls, ball_color);
959 #else
960     // update all balls on core 0 in single-core mode
961     for (int i = 0; i < num_balls; ++i) {
962         updateOneBall(&balls[i]);
963     }
964 
965     drawBallRange(buffer, 0, num_balls, ball_color);
966 #endif
967 
968     // both workers have finished before shared results are read
969     mergeFrameLandings();
970 
971     if (collision_sound_requested[0] ||
972         collision_sound_requested[1]) {
973         playCollisionSound();
974     }
975 }
976 
977 
978 // animation on core 0
979 static PT_THREAD (protothread_anim(struct pt *pt))
980 {
981     // mark beginning of thread
982     PT_BEGIN(pt);
983 
984     while(1) {
985         // wait for the next draw buffer before starting frame timing
986         PT_YIELD_UNTIL(pt, draw_start_signal());
987         gpio_put(FRAME_PROBE, 1);
988 
989         // start time
990         uint64_t start_time = time_us_64();
991 
992         // restore black background and cached pegs into this frame
993         restorePegBackground();
994 
995         // apply captured rotation before either core starts physics
996         applyEncoderChanges();
997         syncBallCount();
998 
999         uint8_t *frame_buffer = (uint8_t *)current_draw_buffer;
1000        uint8_t ball_color = WHITE;
1001 
1002        // measure physics and ball drawing together
1003        gpio_put(PHYSICS_PROBE, 1);
1004        uint64_t update_draw_start = time_us_64();
1005 
1006        updateAndDrawBalls(frame_buffer, ball_color);
```

```
        last_update_draw_us =
            (uint32_t)(time_us_64() - update_draw_start);
        gpio_put(PHYSICS_PROBE, 0);

        drawHist();

        // current ball count, total landings since reset, and time since boot
        drawStats();

        // end time
        uint64_t end_time = time_us_64();

        // elapsed time
        uint64_t elapsed_time = end_time - start_time;
        last_frame_us = (uint32_t)elapsed_time;
        if (elapsed_time > FRAME_RATE_US) ++missed_frames;

        // light the led when frame work takes longer than the target
        if (elapsed_time > FRAME_RATE_US) {
            // set high
            gpio_put(LED_PIN, 1);
        }
        else {
            // set low
            gpio_put(LED_PIN, 0);
        }
        // falling edge marks completion of all work for this frame
        gpio_put(FRAME_PROBE, 0);

        // repeat for the next frame
    } // end frame loop

    PT_END(pt);
} // animation thread


// all vga drawing stays on core 0; only completed ball ranges are drawn

// short tone that fades out
void buildCollisionSound(void)
{
    for (int i = 0; i < SOUND_SAMPLES; i++) {
        float time = (float)i / AUDIO_SAMPLE_RATE; // playback time

        // fade the volume from full to zero
        float envelope = 1.0f - (float)i / (SOUND_SAMPLES - 1);

        float wave = sinf(2.0f * 3.14159265f * TONE_FREQUENCY_HZ * time);
        int sample = (int)DAC_MIDPOINT + (int)(TONE_AMPLITUDE * envelope * wave);

        sound_data[i] = DAC_CONFIG_A | (sample & DAC_SAMPLE_MASK);
    }
}


// set up one dma channel to rewind audio and another to send samples
void initAudioDMA(void)
{
    // claim unused channels because vga also uses dma
    data_chan = dma_claim_unused_channel(true);
    ctrl_chan = dma_claim_unused_channel(true);

    // control channel resets the sample read address
    // then starts the data channel
    dma_channel_config c = dma_channel_get_default_config(ctrl_chan);

    channel_config_set_transfer_data_size(&c, DMA_SIZE_32);
```

```
    channel_config_set_read_increment(&c, false);
    channel_config_set_write_increment(&c, false);
    channel_config_set_chain_to(&c, data_chan);

    dma_channel_configure(
        ctrl_chan,
        &c,
        &dma_hw->ch[data_chan].read_addr, // destination
        &address_pointer,               // source: stored buffer pointer
        1,                              // copy one 32-bit address
        false                           // don't start yet
    );

    // data channel: send 16-bit samples from the buffer to spi
    dma_channel_config c2 = dma_channel_get_default_config(data_chan);

    channel_config_set_transfer_data_size(&c2, DMA_SIZE_16);
    channel_config_set_read_increment(&c2, true);
    channel_config_set_write_increment(&c2, false);

    // set the sample timing from the system clock and audio rate
    int audio_timer = dma_claim_unused_timer(true);
    uint32_t clock_hz = clock_get_hz(clk_sys);
    uint32_t numerator = (uint64_t)AUDIO_SAMPLE_RATE * UINT16_MAX / clock_hz;
    if (numerator == 0) numerator = 1;
    uint32_t denominator = ((uint64_t)clock_hz * numerator
                          + AUDIO_SAMPLE_RATE / 2) / AUDIO_SAMPLE_RATE;
    hard_assert(numerator <= UINT16_MAX && denominator > 0
                && denominator <= UINT16_MAX);
    dma_timer_set_fraction(audio_timer, numerator, denominator);
    channel_config_set_dreq(&c2, dma_get_timer_dreq(audio_timer));

    // do not trigger the control channel afterward
    channel_config_set_chain_to(&c2, data_chan);

    dma_channel_configure(
        data_chan,
        &c2,
        &spi_get_hw(DAC_SPI)->dr, // destination: spi data register
        sound_data,               // source: sound buffer
        SOUND_SAMPLES,            // number of 16-bit transfers
        false                     // don't start yet
    );
}


// ========================================
// === main
// ========================================
int main(){
    // set core voltage before raising processor clock
    vreg_set_voltage(VREG_VOLTAGE_1_35);
    sleep_ms(10);
    set_sys_clock_khz(SYSTEM_CLOCK_KHZ, true) ;

    // initialize serial input and output
    stdio_init_all() ;

    mutex_init(&spawn_random_mutex);
    // seed random values from boot time, which may repeat across restarts
    srand((unsigned int)time_us_64());

    // initialize scope outputs before starting the scheduler
    gpio_init(FRAME_PROBE);
    gpio_set_dir(FRAME_PROBE, GPIO_OUT);
    gpio_put(FRAME_PROBE, 0);
    gpio_init(PHYSICS_PROBE);
    gpio_set_dir(PHYSICS_PROBE, GPIO_OUT);
```

```
    gpio_put(PHYSICS_PROBE, 0);

    // initialize vga
    initVGA() ;

    gpio_init(A_GPIO);
    gpio_init(B_GPIO);
    gpio_init(PULL_UP_GPIO);
    gpio_init(DETECT_PULL_UP);

    // both rotation channels must be inputs
    gpio_set_dir(A_GPIO, GPIO_IN);
    gpio_set_dir(B_GPIO, GPIO_IN);
    gpio_pull_up(A_GPIO);
    gpio_pull_up(B_GPIO);

    // linked button inputs on gpio 22 and gpio 26
    // pull-ups hold the inputs high until the button grounds them
    gpio_set_dir(PULL_UP_GPIO, GPIO_IN);
    gpio_set_dir(DETECT_PULL_UP, GPIO_IN);
    gpio_pull_up(PULL_UP_GPIO);
    gpio_pull_up(DETECT_PULL_UP);

    prevA = gpio_get(A_GPIO) == 0;
    prevB = gpio_get(B_GPIO) == 0;
    prev_detect = gpio_get(DETECT_PULL_UP);
    gpio_set_irq_enabled_with_callback(A_GPIO,
        GPIO_IRQ_EDGE_RISE | GPIO_IRQ_EDGE_FALL, true, encoder_callback);
    gpio_set_irq_enabled(B_GPIO,
        GPIO_IRQ_EDGE_RISE | GPIO_IRQ_EDGE_FALL, true);

    // initialize the led pin
    gpio_init(LED_PIN);
    // configure the led pin as an output
    gpio_set_dir(LED_PIN, GPIO_OUT);
    rotary_encoder_num = max_array;
    // add threads
    pt_add_thread(protothread_serial);
    pt_add_thread(protothread_anim);
    pt_add_thread(poll_encoder_button);

    spi_init(DAC_SPI, DAC_SPI_BAUD);
    spi_set_format(DAC_SPI, 16, 0, 0, SPI_MSB_FIRST);
    gpio_set_function(DAC_CS_GPIO, GPIO_FUNC_SPI);
    gpio_set_function(DAC_SCK_GPIO, GPIO_FUNC_SPI);
    gpio_set_function(DAC_MOSI_GPIO, GPIO_FUNC_SPI);

    // prepare sound samples in memroy
    buildCollisionSound();
    // initballshape();

    // generate pegs and derive histogram geometry before spawning any balls
    generate_pegs();

    initAudioDMA();

    // spawn the configured initial population before the first timed frame
    syncBallCount();
#if USE_SECOND_CORE
    queue_init(&physics_jobs, sizeof(struct PhysicsJob), 1);
    queue_init(&physics_done, sizeof(uint32_t), 1);
    // launch only after geometry, vga, audio, and synchronization are ready
    multicore_launch_core1(core1_entry);
#endif

    // start scheduler
    pt_schedule_start ;
}
```

A.2 VGA Driver: VGA/vga16_graphics_v3.c

Listing 2: Modified VGA driver and drawing routines.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include "pico/stdlib.h"
#include "hardware/pio.h"
#include "hardware/dma.h"
// assembled pio programs
// generated headers for sync and pixel output
#include "hsync.pio.h"
#include "vsync.pio.h"
#include "rgb.pio.h"
// header file
#include "vga16_graphics_v3.h"
// font files
#include "font_glcd.c"
#include "font_ascii_characters.h"
#include "font_rom_237_brl4.h"
#include "font_Arial_round_16x24.h"
#include "font_Grotesk16x32.h"
#include "font_Tiny8.h"
#include <string.h>

/*
================================================================================
some of the source code is derived from adafruit gfx library:
software license agreement (bsd license)

copyright (c) 2012 adafruit industries.  all rights reserved.

redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

- redistributions of source code must retain the above copyright notice,
  this list of conditions and the following disclaimer.
- redistributions in binary form must reproduce the above copyright notice,
  this list of conditions and the following disclaimer in the documentation
  and/or other materials provided with the distribution.

this software is provided by the copyright holders and contributors "as is"
and any express or implied warranties, including, but not limited to, the
implied warranties of merchantability and fitness for a particular purpose
are disclaimed. in no event shall the copyright holder or contributors be
liable for any direct, indirect, incidental, special, exemplary, or
consequential damages (including, but not limited to, procurement of
substitute goods or services; loss of use, data, or profits; or business
interruption) however caused and on any theory of liability, whether in
contract, strict liability, or tort (including negligence or otherwise)
arising in any way out of the use of this software, even if advised of the
possibility of such damage.
================================================================================
many of these fonts were taken from
http://www.rinkydinkelectronics.com/r_fonts.php
the page font license:
all fonts on this page are considered public domain. this means that
you are free to use them as you see fit in any project, commercial or not.
commercial projects will still need a commercial license for any libraries used.
================================================================================
- gpio 16 ---> vga hsync
- gpio 17 ---> vga vsync
```

```
- gpio 18 ---> vga green lo-bit --> 470 ohm resistor -->  vga_green
- gpio 19 ---> vga green hi_bit --> 330 ohm resistor -->  vga_green
- gpio 20 ---> 330 ohm resistor ---> vga-blue
- gpio 21 ---> 330 ohm resistor ---> vga-red
- rp2040 gnd ---> vga-gnd

new text commands are re-entrant
drawpixel is faster
*/

// vga timing constants
#define H_ACTIVE   655    // (active + frontporch - 1) - one cycle delay for mov
#define V_ACTIVE   479    // (active - 1)
#define RGB_ACTIVE 639    // 640 pixel clocks minus one
// #define rgb_active 639 // change to this if 1 pixel/byte

// length of the pixel array, and number of dma transfers
// monochrome framebuffer: eight pixels per byte
#define VGA_MONO_WIDTH 640
#define VGA_MONO_HEIGHT 480
#define VGA_BYTES_PER_ROW (VGA_MONO_WIDTH / 8)
#define VGA_BUFFER_COUNT (VGA_BYTES_PER_ROW * VGA_MONO_HEIGHT)

// ==============================
// !!!=========================!!!
// define exactly one of
// double_buffer_60, double_buffer_30, double_buffer_none
//double-buffered!
//turn off second buffer to save memory on rp2040
// ==============================
#define DOUBLE_BUFFER_60
//#define double_buffer_30
//#define double_buffer_none
// !!!~=========================!!!
// ==============================

// packed pixel data sent to pio by dma
// pointer to the start of this buffer
// starts filled with zeroes for a black screen
unsigned char vga_buffer_0[VGA_BUFFER_COUNT] __attribute__((aligned(4)));
char * pointer_vga_buffer_0 = &vga_buffer_0[0] ;
//
// only define second buffer if necessary
#ifndef DOUBLE_BUFFER_NONE
  unsigned char vga_buffer_1[VGA_BUFFER_COUNT] __attribute__((aligned(4)));
  char * pointer_vga_buffer_1 = &vga_buffer_1[0] ;
#endif
//
// the actual pointer taken from a table defined below
// and loaded by a dma channel
char * current_draw_buffer ;
char * pointer_current_display_buffer ;

// define the buffer order arrays to be cycled thru by dma
// must be aligned to use dma hardware looping
char * draw_buffer[4]  __attribute__ ((aligned (16))) ;
char * pointer_display_buffer[4] __attribute__ ((aligned (16)));
int    start_flag_array[4] __attribute__ ((aligned (16)));

// dma sets this when it is time to draw
int start_flag = 0 ;
// used to signal buffer type to thread
int buffer_type ;

// unused masks from the older 4-bit pixel layout
#define TOPMASK 0b00001111
#define BOTTOMMASK 0b11110000

```

```
128 // for drawline
129 #define swap(a, b) { short t = a; a = b; b = t; }
130
131 // legacy text spacing
132 #define tabspace 4 // pixels added for a tab
133
134 // read font data from flash
135 #define pgm_read_byte(addr) (*(const unsigned char *)(addr))
136 #define pgm_read_short(addr) (*(const unsigned short *)(addr))
137
138 // shared settings for the legacy text routines
139 unsigned short cursor_y, cursor_x, textsize ;
140 char textcolor, textbgcolor, wrap;
141
142 // screen width/height
143 #define _width 640
144 #define _height 480
145
146 // set up video buffers, pio output, and the dma buffer-swap chain
147 void initVGA() {
148     // use pio0 for sync signals and pixel output
149     PIO pio = pio0;
150
151     // load all three programs into shared pio instruction memory
152     // keep their offsets for setup, with 32 instructions available in total
153     uint hsync_offset = pio_add_program(pio, &hsync_program);
154     uint vsync_offset = pio_add_program(pio, &vsync_program);
155     uint rgb_offset = pio_add_program(pio, &rgb_program);
156
157     // manually select a few state machines from pio instance pio0
158     // void pio_sm_claim (pio pio, uint sm)
159     uint hsync_sm = 0;
160     uint vsync_sm = 1;
161     uint rgb_sm = 2;
162     pio_sm_claim (pio, hsync_sm);
163     pio_sm_claim (pio, vsync_sm);
164     pio_sm_claim (pio, rgb_sm);
165
166     // set up each state machine using its generated pio helper
167     hsync_program_init(pio, hsync_sm, hsync_offset, HSYNC);
168     vsync_program_init(pio, vsync_sm, vsync_offset, VSYNC);
169     rgb_program_init(pio, rgb_sm, rgb_offset, LO_GRN);
170
171     //
        ////////////////////////////////////////////////////////////////////////////////////////////////////
172     // ============================== pio dma channels
        ================================================
173     //
        ////////////////////////////////////////////////////////////////////////////////////////////////////
174     // initialize
175     current_draw_buffer = vga_buffer_0 ;
176
177     // define arrays used by dma channels to automatically swap buffers
178     // advance buffer choices when the pixel dma transfer finishes
179     #ifdef DOUBLE_BUFFER_60
180       buffer_type = 1;
181       // alternate draw/display
182       draw_buffer[0] = vga_buffer_1 ;
183       draw_buffer[1] = vga_buffer_0 ;
184       draw_buffer[2] = vga_buffer_1 ;
185       draw_buffer[3] = vga_buffer_0 ;
186       //
187       pointer_display_buffer[0] = pointer_vga_buffer_0 ;
188       pointer_display_buffer[1] = pointer_vga_buffer_1 ;
189       pointer_display_buffer[2] = pointer_vga_buffer_0 ;
190       pointer_display_buffer[3] = pointer_vga_buffer_1 ;
```

```
191       // start draw is valid on every video frame
192       start_flag_array[0] = 1 ;
193       start_flag_array[1] = 1 ;
194       start_flag_array[2] = 1 ;
195       start_flag_array[3] = 1 ;
196     #endif
197
198     #ifdef DOUBLE_BUFFER_30
199       buffer_type = 2;
200       draw_buffer[0] = vga_buffer_0 ;
201       draw_buffer[1] = vga_buffer_0 ;
202       draw_buffer[2] = vga_buffer_1 ;
203       draw_buffer[3] = vga_buffer_1 ;
204       //
205       pointer_display_buffer[0] = pointer_vga_buffer_1 ;
206       pointer_display_buffer[1]= pointer_vga_buffer_1 ;
207       pointer_display_buffer[2] = pointer_vga_buffer_0  ;
208       pointer_display_buffer[3] = pointer_vga_buffer_0 ;
209       // start draw is valid on every other video frame
210       start_flag_array[0] = 2 ;
211       start_flag_array[1] = 0 ;
212       start_flag_array[2] = 2 ;
213       start_flag_array[3] = 0 ;
214     #endif
215
216     #ifdef DOUBLE_BUFFER_NONE
217       buffer_type = 3;
218       draw_buffer[0] = vga_buffer_0 ;
219       draw_buffer[1] = vga_buffer_0 ;
220       draw_buffer[2] = vga_buffer_0 ;
221       draw_buffer[3] = vga_buffer_0 ;
222       //
223       pointer_display_buffer[0] = pointer_vga_buffer_0;
224       pointer_display_buffer[1]= pointer_vga_buffer_0 ;
225       pointer_display_buffer[2] = pointer_vga_buffer_0  ;
226       pointer_display_buffer[3] = pointer_vga_buffer_0 ;
227       // single buffer shared by drawing and display
228       start_flag_array[0] = 3 ;
229       start_flag_array[1] = 3 ;
230       start_flag_array[2] = 3 ;
231       start_flag_array[3] = 3 ;
232     #endif
233
234     // dma channels -
235     // data_chan sends color data to output pio
236     // disp_chan sets the current display buffer
237     // draw_chan sets the current drawing buffer
238     // start_chan signals the draw thread to start drawing
239     int rgb_data_chan = dma_claim_unused_channel(true); //data
240     int set_disp_chan = dma_claim_unused_channel(true); // reset pointer to current displqy
241     int set_draw_chan = dma_claim_unused_channel(true); //
242     int set_start_chan = dma_claim_unused_channel(true); //
243
244     // optional higher priority for video dma
245     #define rgb_high_priority false
246
247     // send packed pixel bytes to the rgb state machine
248     dma_channel_config c0 = dma_channel_get_default_config(rgb_data_chan);  // default
            configs
249     channel_config_set_transfer_data_size(&c0, DMA_SIZE_8);              // 8-bit txfers
250     channel_config_set_read_increment(&c0, true);                        // yes read
            incrementing
251     channel_config_set_write_increment(&c0, false);                      // no write
            incrementing
252     channel_config_set_dreq(&c0, DREQ_PIO0_TX2) ;                        // dreq_pio0_tx2
            pacing (fifo)
253     channel_config_set_chain_to(&c0, set_disp_chan);                        // chain to
            other channel
```

```
254     channel_config_set_high_priority (&c0, rgb_high_priority) ;
255
256     dma_channel_configure(
257         rgb_data_chan,                // dma channel
258         &c0,                        // channel settings
259         &pio->txf[rgb_sm],          // write address (rgb pio tx fifo)
260         &vga_buffer_0,            // the initial read address (pixel color array)
261         VGA_BUFFER_COUNT,           // number of pixel bytes to send
262         false                       // start later
263     );
264
265     // channel set_disp_chan (reconfigures the first channel)
266     // sets pointer_display_buffer from a table
267     dma_channel_config c1 = dma_channel_get_default_config(set_disp_chan);   // default
            configs
268     channel_config_set_transfer_data_size(&c1, DMA_SIZE_32);              // 32-bit txfers
269     channel_config_set_read_increment(&c1, true);                        // read
            incrementing
270     channel_config_set_write_increment(&c1, false);                       // no write
            incrementing
271     channel_config_set_chain_to(&c1, set_draw_chan);                       // chain to other
             channel
272     channel_config_set_ring (&c1, false, 4); // 16 byte table
273
274     dma_channel_configure(
275         set_disp_chan,                          // dma channel
276         &c1,                                  // channel settings
277         &dma_hw->ch[rgb_data_chan].read_addr,  // update the pixel dma read address
278         pointer_display_buffer,              // read the next table entry
279         1,                                    // copy one 4-byte entry
280         false                                 // start later
281     );
282
283     // send the draw pointer to the current draw pointer
284     // sets current_draw_buffer from a table
285     c1 = dma_channel_get_default_config(set_draw_chan);   // default configs
286     channel_config_set_transfer_data_size(&c1, DMA_SIZE_32);              // 32-bit txfers
287     channel_config_set_read_increment(&c1, true);                        // read
            incrementing
288     channel_config_set_write_increment(&c1, false);                       // no write
            incrementing
289     channel_config_set_chain_to(&c1, set_start_chan);                       // chain to
            other channel
290     channel_config_set_ring (&c1, false, 4); // 16 byte table
291
292     dma_channel_configure(
293         set_draw_chan,                          // dma channel
294         &c1,                                  // channel settings
295         &current_draw_buffer,                 // destination variable
296         draw_buffer,                         // read the next table entry
297         1,                                    // copy one 4-byte entry
298         false                                 // start later
299     );
300
301     // send the start signal to a thread
302     // sets a variable from a table
303      c1 = dma_channel_get_default_config(set_start_chan);   // default configs
304     channel_config_set_transfer_data_size(&c1, DMA_SIZE_32);              // 32-bit txfers
305     channel_config_set_read_increment(&c1, true);                        // read
            incrementing
306     channel_config_set_write_increment(&c1, false);                       // no write
            incrementing
307     channel_config_set_chain_to(&c1, rgb_data_chan);                       // chain to other
             channel
308     channel_config_set_ring (&c1, false, 4); // 16 byte table
309
310     dma_channel_configure(
311         set_start_chan,                          // dma channel
```

```
        &c1,                                // channel settings
        &start_flag,                       // destination variable
        start_flag_array,                       // read the next table entry
        1,                                 // copy one 4-byte entry
        false                              // start later
    );

    //
        ////////////////////////////////////////////////////////////////////////////////////////////////////

    //
        ////////////////////////////////////////////////////////////////////////////////////////////////////


    // supply timing counters for the initial pio pull instructions
    pio_sm_put_blocking(pio, hsync_sm, H_ACTIVE);
    pio_sm_put_blocking(pio, vsync_sm, V_ACTIVE);

    // prime the rgb state machine while it is disabled
    pio_sm_put_blocking(pio, rgb_sm, RGB_ACTIVE);

    // load the counter from the fifo into osr
    pio_sm_exec(pio, rgb_sm, pio_encode_pull(false, true));

    // keep the scanline counter in isr
    pio_sm_exec(pio, rgb_sm, pio_encode_mov(pio_isr, pio_osr));

    // discard the counter from osr so the next out fetches pixel data
    pio_sm_exec(pio, rgb_sm, pio_encode_out(pio_null, 32));


    // start all three state machines together
    pio_enable_sm_mask_in_sync(pio, ((1u << hsync_sm) | (1u << vsync_sm) | (1u << rgb_sm)));

    // start the pixel dma chain for continuous screen output
    dma_start_channel_mask((1u << rgb_data_chan)) ;
}

//
    ////////////////////////////////////////////////////////////////////////////////////////////////////

// ============================== drawing routines
    ================================================
//
    ////////////////////////////////////////////////////////////////////////////////////////////////////


// update one monochrome pixel in the drawing buffer
// zero means black, any other color means white
void drawPixel(short x, short y, char color)
{
    if (x < 0 || x >= VGA_MONO_WIDTH ||
        y < 0 || y >= VGA_MONO_HEIGHT)
        return;

    uint8_t *buffer = (uint8_t *)current_draw_buffer;

    // eight pixels per byte
    int index = y * VGA_BYTES_PER_ROW + (x >> 3);

    // select the pixel's bit within that byte
    uint8_t mask = (uint8_t)(1u << (x & 7));

    if (color != 0) {
        buffer[index] |= mask;           // set pixel to white
    } else {
        buffer[index] &= (uint8_t)~mask; // set pixel to black
    }
```

```
371 }
372
373 // count the eight live cells around this cell
374 int checkNeighbors(short x, short y) {
375     char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
376
377     return (isAlive(x-1, y-1) + isAlive(x, y-1) + isAlive(x+1, y-1) +
378             isAlive(x-1, y) + isAlive(x+1, y) +
379             isAlive(x-1, y+1) + isAlive(x, y+1) + isAlive(x+1, y+1));
380 }
381
382 // draw a cell as a 2 by 2 pixel square
383 void drawCell(short x, short y, char color) {
384
385     drawPixel(x<<1, y<<1, color) ;
386     drawPixel((x<<1) + 1, (y<<1), color) ;
387     drawPixel((x<<1), (y<<1) + 1, color) ;
388     drawPixel((x<<1) + 1, (y<<1) + 1, color) ;
389
390 }
391
392 // check the low color bit at the cell's top-left pixel
393 int isAlive(short x, short y) {
394     return (readPixel(x<<1, y<<1) & 1) ;
395 }
396
397 // vertical line
398 void drawVLine(short x, short y, short h, char color) {
399     for (short i=y; i<(y+h); i++) {
400         drawPixel(x, i, color) ;
401     }
402 }
403
404 // horizontal line
405 // use individual pixels at the ends
406 // fill whole bytes in the middle for speed
407 void drawHLine(int x, int y, int w, char color)
408 {
409     if (y < 0 || y >= VGA_MONO_HEIGHT ||
410         w <= 0 || x >= VGA_MONO_WIDTH)
411         return;
412
413     // clip the line at the left edge
414     if (x < 0) {
415         w += x;
416         x = 0;
417     }
418
419     if (w <= 0)
420         return;
421
422     // clip the line at the rigth edge
423     if (w > VGA_MONO_WIDTH - x)
424         w = VGA_MONO_WIDTH - x;
425
426     // draw individual pixels until reaching a byte boundary
427     while ((x & 7) != 0 && w > 0) {
428         drawPixel(x, y, color);
429         x++;
430         w--;
431     }
432
433     // fill complete bytes, eight pixles per byte
434     int whole_bytes = w / 8;
435
436     if (whole_bytes > 0) {
437         memset(current_draw_buffer +
438                    y * VGA_BYTES_PER_ROW + (x >> 3),
```

```
                color != 0 ? 0xFF : 0x00,
                whole_bytes);

        x += whole_bytes * 8;
        w -= whole_bytes * 8;
    }

    // draw the remaining pixels without touching their neighbors
    while (w > 0) {
        drawPixel(x, y, color);
        x++;
        w--;
    }
}

// general line drawing
// bresenham's algorithm - thx wikipedia and thx bruce!
void drawLine(short x0, short y0, short x1, short y1, char color) {
/* line between the two endpoints using integer error steps
 * x increases rightward, y increases downward
 * zero draws black, any other color draws white
 */
      short steep = abs(y1 - y0) > abs(x1 - x0);
      if (steep) {
        swap(x0, y0);
        swap(x1, y1);
      }

      if (x0 > x1) {
        swap(x0, x1);
        swap(y0, y1);
      }

      short dx, dy;
      dx = x1 - x0;
      dy = abs(y1 - y0);

      short err = dx / 2;
      short ystep;

      if (y0 < y1) {
        ystep = 1;
      } else {
        ystep = -1;
      }

      for (; x0<=x1; x0++) {
        if (steep) {
          drawPixel(y0, x0, color);
        } else {
          drawPixel(x0, y0, color);
        }
        err -= dy;
        if (err < 0) {
          y0 += ystep;
          err += dx;
        }
      }
}

// draw a rectangle
void drawRect(short x, short y, short w, short h, char color) {
/* outline from top-left x and y, with width w and height h */
  drawHLine(x, y, w, color);
  drawHLine(x, y+h-1, w, color);
  drawVLine(x, y, h, color);
  drawVLine(x+w-1, y, h, color);
}
```

```
507
508 void drawCircle(short x0, short y0, short r, char color) {
509 /* circle outline centered at x0 and y0, mirrored across eight directions */
510   short f = 1 - r;
511   short ddF_x = 1;
512   short ddF_y = -2 * r;
513   short x = 0;
514   short y = r;
515
516   drawPixel(x0  , y0+r, color);
517   drawPixel(x0  , y0-r, color);
518   drawPixel(x0+r, y0  , color);
519   drawPixel(x0-r, y0  , color);
520
521   while (x<y) {
522     if (f >= 0) {
523       y--;
524       ddF_y += 2;
525       f += ddF_y;
526     }
527     x++;
528     ddF_x += 2;
529     f += ddF_x;
530
531     drawPixel(x0 + x, y0 + y, color);
532     drawPixel(x0 - x, y0 + y, color);
533     drawPixel(x0 + x, y0 - y, color);
534     drawPixel(x0 - x, y0 - y, color);
535     drawPixel(x0 + y, y0 + x, color);
536     drawPixel(x0 - y, y0 + x, color);
537     drawPixel(x0 + y, y0 - x, color);
538     drawPixel(x0 - y, y0 - x, color);
539   }
540 }
541
542 void drawCircleHelper( short x0, short y0, short r, unsigned char cornername, char color) {
543 // draw the circle quarters selected by the corner bits
544   short f     = 1 - r;
545   short ddF_x = 1;
546   short ddF_y = -2 * r;
547   short x     = 0;
548   short y     = r;
549
550   while (x<y) {
551     if (f >= 0) {
552       y--;
553       ddF_y += 2;
554       f     += ddF_y;
555     }
556     x++;
557     ddF_x += 2;
558     f     += ddF_x;
559     if (cornername & 0x4) {
560       drawPixel(x0 + x, y0 + y, color);
561       drawPixel(x0 + y, y0 + x, color);
562     }
563     if (cornername & 0x2) {
564       drawPixel(x0 + x, y0 - y, color);
565       drawPixel(x0 + y, y0 - x, color);
566     }
567     if (cornername & 0x8) {
568       drawPixel(x0 - y, y0 + x, color);
569       drawPixel(x0 - x, y0 + y, color);
570     }
571     if (cornername & 0x1) {
572       drawPixel(x0 - y, y0 - x, color);
573       drawPixel(x0 - x, y0 - y, color);
574     }
```

```
  }
}

// ================================================
// int sqrt from https://github.com/chmike/fpsqrt/blob/master/fpsqrt.c
int32_t sqrt_i32(int32_t v) {
    uint32_t b = 1<<30, q = 0, r = v;
    while (b > r)
        b >>= 2;
    while( b > 0 ) {
        uint32_t t = q + b;
        q >>= 1;
        if( r >= t ) {
            r -= t;
            q += b;
        }
        b >>= 2;
    }
    return q;
}
// ============================
// fill a circle
// uses simple (but fast) sqrt algorithm
void fillCircle(short x0, short y0, short r, char color) {
  // slightly expand the squared radius for the pixel outline
  int r2 = r * r + r;
  if((y0-r < 0) || (y0+r > 479)) return ;
  for(int i=0; i<=r; i++){
    // find the half-width for this circle row
    int dx = sqrt_i32(r2 - i*i) ;
    // drawhline(int x, int y, int w, char color)
    drawHLine(x0-dx, y0+(i), 2*dx, color) ;
    drawHLine(x0-dx, y0-(i), 2*dx, color) ;
  }
}
// ================================================
// legacy routine
// void fillcircle(short x0, short y0, short r, char color) {
// /* draw a filled circle with center (x0,y0) and radius r, with given color
//  * parameters:
//  *      x0: x-coordinate of center of circle the top-left of the screen
//  *          has x-coordinate 0 and increases to the right
//  *      y0: y-coordinate of center of circle the top-left of the screen
//  *          has y-coordinate 0 and increases to the bottom
//  *      r:  radius of circle
//  *      color: 16-bit color value for the circle
//  * returns: nothing
//  */

//   drawvline(x0, y0-r, 2*r+1, color);
//   fillcirclehelper(x0, y0, r, 3, 0, color);
// }

void fillCircleHelper(short x0, short y0, short r, unsigned char cornername, short delta,
    char color) {
// fill selected circle halves with vertical lines, extended by delta
  short f     = 1 - r;
  short ddF_x = 1;
  short ddF_y = -2 * r;
  short x     = 0;
  short y     = r;

  while (x<y) {
    if (f >= 0) {
      y--;
      ddF_y += 2;
      f     += ddF_y;
    }
```

```
642     x++;
643     ddF_x += 2;
644     f     += ddF_x;
645 
646     if (cornername & 0x1) {
647       drawVLine(x0+x, y0-y, 2*y+1+delta, color);
648       drawVLine(x0+y, y0-x, 2*x+1+delta, color);
649     }
650     if (cornername & 0x2) {
651       drawVLine(x0-x, y0-y, 2*y+1+delta, color);
652       drawVLine(x0-y, y0-x, 2*x+1+delta, color);
653     }
654   }
655 }
656 
657 
658 // draw a rounded rectangle
659 void drawRoundRect(short x, short y, short w, short h, short r, char color) {
660 /* rectangle outline with rounded corners of radius r */
661   // smarter version
662   drawHLine(x+r  , y    , w-2*r, color); // top
663   drawHLine(x+r  , y+h-1, w-2*r, color); // bottom
664   drawVLine(x    , y+r  , h-2*r, color); // left
665   drawVLine(x+w-1, y+r  , h-2*r, color); // right
666   // draw four corners
667   drawCircleHelper(x+r    , y+r    , r, 1, color);
668   drawCircleHelper(x+w-r-1, y+r    , r, 2, color);
669   drawCircleHelper(x+w-r-1, y+h-r-1, r, 4, color);
670   drawCircleHelper(x+r    , y+h-r-1, r, 8, color);
671 }
672 
673 // ***************************************************
674 void fillRoundRect(short x, short y, short w, short h, short r, char color) {
675   // smarter version
676   fillRect(x, y+r, w, h-2*r, color);
677   fillRect(x+r, y, w-2*r, r, color);
678   fillRect(x+r, y+h-r, w-2*r, r, color);
679 
680   // draw four corners
681   fillCircle(x+w-r, y+r, r-1, color);
682   fillCircle(x+r  , y+r, r-1, color);
683   fillCircle(x+w-r, y+h-r, r-1, color);
684   fillCircle(x+r,   y+h-r, r-1, color);
685 }
686 
687 // // ***************************************************
688 // // fill a rounded rectangle
689 // void fillroundrect(short x, short y, short w, short h, short r, char color) {
690 //   // smarter version
691 //   fillrect(x+r, y, w-2*r, h, color);
692 
693 //   // draw four corners
694 //   fillcirclehelper(x+w-r-1, y+r, r, 1, h-2*r-1, color);
695 //   fillcirclehelper(x+r    , y+r, r, 2, h-2*r-1, color);
696 // }
697 
698 
699 // fill a rectangle
700 void fillRect(short x, short y, short w, short h, char color) {
701 /* fill horizontal rows from the top-left corner using width w and height h */
702    if((y + h - 1) >= _height) h = _height - y - 1;
703 
704   for(int j=y; j<(y+h); j++) {
705     drawHLine(x, j, w, color) ;
706   }
707 }
708 
709 ////////////////////////////////////////////////////////////////////////
```

```
// copied with minor mods from
// https://ece4760.github.io/projects/fall2023/av522_dy245/code.html
////////////////////////////////////////////////////////////////////
// draw a filled triangle
// fill horizontal rows between the triangle edges
// (see https://en.wikipedia.org/wiki/rasterisation)
void fillTri(float x0, float y0, float x1, float y1, float x2, float y2, char color) {
  //
  // sort verts so y0 <= y1 <= y2 (p0 = top, p1 = middle, p2 = bottom)
  if (y1 < y0) {
    swap(x0, x1);
    swap(y0, y1);
  }
  if (y2 < y0) {
    swap(x0, x2);
    swap(y0, y2);
  }
  if (y2 < y1) {
    swap(x1, x2);
    swap(y1, y2);
  }

  // calculate floating-point edge slopes, avoiding division by zero
  float dxdy_01 = y1 == y0 ? 0 :  (x1 - x0) / (y1 - y0);
  float dxdy_02 = y2 == y0 ? 0 :  (x2 - x0) / (y2 - y0);
  float dxdy_12 = y2 == y1 ? 0 :  (x2 - x1) / (y2 - y1);
  // disabled depth slopes
  //s15x16 dzdy_01 = y1 == y0 ? 0 : divs15x16(int_to_s15x16(z1 - z0), y1 - y0);
 // s15x16 dzdy_02 = y2 == y0 ? 0 : divs15x16(int_to_s15x16(z2 - z0), y2 - y0);
 // s15x16 dzdy_12 = y2 == y1 ? 0 : divs15x16(int_to_s15x16(z2 - z1), y2 - y1);

  // figure out whether p1 is on the left or right side of the triangle
  bool flat_top = (y0 == y1);
  bool flat_bottom = (y1 == y2);
  bool p1_is_left = flat_top ? (x0 > x1) : (dxdy_02 > dxdy_01);

  // starting at p0, we draw horizontal scanlines (from x_left to x_right, at height y)
  float x_left = x0;
  float x_right = x0;
  float y = y0;

  // unused depth interpolation below
  //s15x16 z_left = int_to_s15x16(z0) + zeropt5;
  //s15x16 z_right = int_to_s15x16(z0) + zeropt5;

  // x_left and x_right are moved based on slopes of left/right edges
  float dx_left, dx_right;
  float dz_left, dz_right;
  if (p1_is_left) {
    dx_left = dxdy_01;
    dx_right = dxdy_02;
   // dz_left = dzdy_01;
   // dz_right = dzdy_02;
  } else {
    dx_left = dxdy_02;
    dx_right = dxdy_01;
    //dz_left = dzdy_02;
    //dz_right = dzdy_01;
  }

  // macro function to move the scanline down, and update its endpoints
  #define moveScanline() {\
    y += 1 ;\
    x_left += dx_left;\
    x_right += dx_right;\
  }
    //z_left += dz_left;\
   // z_right += dz_right;\
```

```
778   }
779
780   // draw top half of triangle; skipped for flat top case
781   while (y < y1) {
782     //drawscanline((short)fix2int15(y), (short)fix2int15(x_left), (short)fix2int15(x_right),
          z_left, z_right, color);
783     drawHLine((int) (x_left), (int) (y), abs((int) (x_right-x_left)), color);
784
785     moveScanline();
786   }
787
788   // flat bottom triangles skip the rest
789   if (flat_bottom)
790     return;
791
792   // reconfigure one end of the scanline so it goes from p1 to p2
793   if (p1_is_left) {
794     x_left = x1;
795     dx_left = dxdy_12;
796     //z_left = int2fix15(z1) + zeropt5;
797    // dz_left = dzdy_12;
798   } else {
799     x_right = x1;
800     dx_right = dxdy_12;
801    // z_right = int2fix15(z1) + zeropt5;
802    // dz_right = dzdy_12;
803   }
804
805   // draw horizontal line through p1 (middle)
806   //drawscanline((short)fix2int15(y), (short)fix2int15(x_left), (short)fix2int15(x_right),
        z_left, z_right, color);
807   drawHLine((int) (x_left), (int) (y), (int) (x_right-x_left), color);
808   // draw bottom half of triangle; skipped for flat bottom case
809   while (y < y2) {
810     moveScanline();
811     //drawscanline((short)fix2int15(y), (short)fix2int15(x_left), (short)fix2int15(x_right),
          z_left, z_right, color);
812     drawHLine((int) (x_left), (int) (y), (int) (x_right-x_left), color);
813   }
814 }
815
816 // ===========================================
817 // end copied code
818 // ===========================================
819 // application builds an array of
820 // short point_list[numlines][2]
821
822 // connect each point to the previous point
823 void drawMultiLine(int num_lines,  short point_list[][2], char color){
824   for(int i=1; i<num_lines; i++){
825     drawLine(point_list[i-1][0], point_list[i-1][1], point_list[i][0], point_list[i][1],
          color);
826   }
827 }
828
829 // === text stuff ============================
830 // note!!!
831 // legacy routine
832 // shared text settings, not safe for overlapping calls
833 // see below for the new text routines
834 // kept for older code
835 // draw a character
836 void drawChar(short x, short y, unsigned char c, char color, char bg, unsigned char size) {
837     char i, j;
838   if((x >= _width)            || // clip right
839      (y >= _height)           || // clip bottom
840      ((x + 6 * size - 1) < 0) || // clip left
841      ((y + 8 * size - 1) < 0))   // clip top
```

```
842     return;
843
844   for (i=0; i<6; i++ ) {
845     unsigned char line;
846     if (i == 5)
847       line = 0x0;
848     else
849       line = pgm_read_byte(font+(c*5)+i);
850     for ( j = 0; j<8; j++) {
851       if (line & 0x1) {
852         if (size == 1) // default size
853           drawPixel(x+i, y+j, color);
854         else {  // big size
855           fillRect(x+(i*size), y+(j*size), size, size, color);
856         }
857       } else if (bg != color) {
858         if (size == 1) // default size
859           drawPixel(x+i, y+j, bg);
860         else {  // big size
861           fillRect(x+i*size, y+j*size, size, size, bg);
862         }
863       }
864       line >>= 1;
865     }
866   }
867 }
868 // legacy routine
869 inline void setCursor(short x, short y) {
870 /* set the starting position for legacy text */
871   cursor_x = x;
872   cursor_y = y;
873 }
874 // legacy routine
875 inline void setTextSize(unsigned char s) {
876 /* set legacy text scale, with a minimum of 1 */
877   textsize = (s > 0) ? s : 1;
878 }
879 // legacy routine
880 inline void setTextColor(char c) {
881   // match background and foreground to skip background pixels
882   // used by the legacy character routines
883   textcolor = textbgcolor = c;
884 }
885 // legacy routine
886 inline void setTextColor2(char c, char b) {
887 /* store foreground and background colors for legacy text */
888   textcolor   = c;
889   textbgcolor = b;
890 }
891 // legacy routine
892 inline void setTextWrap(char w) {
893   wrap = w;
894 }
895
896 // legacy routine
897 void tft_write(unsigned char c){
898   if (c == '\n') {
899     cursor_y += textsize*8;
900     cursor_x  = 0;
901   } else if (c == '\r') {
902     // skip em
903   } else if (c == '\t'){
904       int new_x = cursor_x + tabspace;
905       if (new_x < _width){
906           cursor_x = new_x;
907       }
908   } else {
909     drawChar(cursor_x, cursor_y, c, textcolor, textbgcolor, textsize);
```

```
    cursor_x += textsize*6;
    if (wrap && (cursor_x > (_width - textsize*6))) {
      cursor_y += textsize*8;
      cursor_x = 0;
    }
  }
}
// legacy routine
inline void writeString(char* str){
/* draw using the shared cursor, color, and size settings */
    while (*str){
        tft_write(*str++);
    }
}

//================================================
// added 10/16/2023 brl4
// legacy routine
inline void setTextColorBig(char color, char background) {
/* store foreground and background colors for legacy text */
  textcolor   = color;
  textbgcolor = background;
}
//================================================
// added 10/11/2023 brl4
// draw a character
// legacy routine
void drawCharBig(short x, short y, unsigned char c, char color, char bg) {
  char i, j ;
  unsigned char line;
  for (i=0; i<15; i++ ) {
    line = pgm_read_byte(bigFont+((int)c*16)+i);
    for ( j = 0; j<8; j++) {
      if (line & 0x80) {
        drawPixel(x+j, y+i, color);
      } else if (bg!=color){
        drawPixel(x+j, y+i, bg);
      }
      line <<= 1;
    }
  }
}
// legacy routine
inline void writeStringBig(char* str){
/* draw using the shared cursor, color, and size settings */
    while (*str){
      char c = *str++;
        drawCharBig(cursor_x, cursor_y, c, textcolor, textbgcolor);
        cursor_x += 8 ;
    }
}
// legacy routine
inline void writeStringBold(char* str){
/* draw using the shared cursor, color, and size settings */
   /* draw using the shared cursor, color, and size settings */
    char temp_bg ;
    temp_bg = textbgcolor;
    while (*str){
        char c = *str++;
        drawChar(cursor_x, cursor_y, c, textcolor, textbgcolor, textsize);
        drawChar(cursor_x+1, cursor_y, c, textcolor, textcolor, textsize);
        cursor_x += 7 * textsize ;
    }
    textbgcolor = temp_bg ;
}

// ================================================
//re-entrant text -- >>use these!<<
```

```
978 //
979 // //glcd font adafruit and hunter
980 // returns num chars drawn
981 int drawTextGLCD(short x, short y, char *str, char color, char bgcolor)
982 {
983     int char_count = 0;
984     int cursor_x = x;
985 
986     // each character occupies 6 columns and 8 rows
987     if (cursor_x < 0 || y < 0 || y > VGA_MONO_HEIGHT - 8)
988         return 0;
989 
990     while (*str != '\0') {
991         if (cursor_x > VGA_MONO_WIDTH - 6)
992             break;
993 
994         unsigned char ch = (unsigned char)*str++;
995 
996         // five character columns plus one spacing column
997         for (int col = 0; col < 6; col++) {
998             uint8_t column_bits = 0;
999 
1000             if (col < 5) {
1001                 column_bits = pgm_read_byte( font + (unsigned int)ch * 5 + col );
1002             }
1003 
1004             for (int row = 0; row < 8; row++) {
1005                 char pixel_color = (column_bits & (1u << row)) ? color : bgcolor;
1006 
1007                 drawPixel(cursor_x + col,
1008                           y + row,
1009                           pixel_color);
1010             }
1011         }
1012 
1013         cursor_x += 6;
1014         char_count++;
1015     }
1016 
1017     return char_count;
1018 }
1019 // ascii from designed by: david perez de la cruz,and ed lau
1020 // see: https://people.ece.cornell.edu/land/courses/ece4760/finalprojects/s2005/dp93/index.
     html
1021 //
1022 // legacy packed-color text, using three bytes per six-pixel row
1023 int drawTextAscii(short x, short y, char * str, char color, char bgcolor){
1024   // locate the starting buffer position
1025   int char_count = 0 ;
1026   char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
1027   // error check
1028   if(x<0 | y<0 | y>470) return 0;
1029   // precompute the four foreground and background pixel pairs
1030   char pix_value[4] =
1031     {(bgcolor<<4 | bgcolor), (color<<4 | bgcolor), (bgcolor<<4 | color), (color<<4 | color)
          };
1032   // one row of the character bitmap
1033   unsigned char line;
1034   while (*str){
1035     if((x+6 > 639)) return char_count ;
1036     // s
1037     char c = (*str++) ;
1038     char_count++ ;
1039     for (int i=0; i<7; i++ ) {
1040       line = pgm_read_byte(asciifont+((int)c*7)+i);
1041       // pack two pixels per byte using the older color layout
1042       // select each pixel pair from the lookup table
1043         *(draw_loc+i*320) =   pix_value[(line>>6) & 0x03] ;
```

```
1044          *(draw_loc+i*320+1) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
1045          *(draw_loc+i*320+2) = pix_value[(line>>2) & 0x03] ;
1046          //*(draw_loc+i*320+3) = pix_value[(line) & 0x03] ;
1047        }
1048        draw_loc += 3 ;
1049        x += 6 ;
1050      }
1051      return char_count ;
1052    }
1053
1054    //
1055    // tinyfont from http://www.rinkydinkelectronics.com/r_fonts.php
1056    //
1057    // legacy packed-color text, eight rows and eight pixels per character
1058    int drawTextTiny8(short x, short y, char * str, char color, char bgcolor){
1059      // locate the starting buffer position
1060      int char_count = 0 ;
1061      char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
1062      // error check
1063      if(x<0 | y<0 | y>470) return 0;
1064      // precompute the four foreground and background pixel pairs
1065      char pix_value[4] =
1066        {(bgcolor<<4 | bgcolor), (color<<4 | bgcolor), (bgcolor<<4 | color), (color<<4 | color)
             };
1067      // one row of the character bitmap
1068      unsigned char line;
1069      while (*str){
1070        if((x+8 > 639)) return char_count ;
1071        // subtract 32 since the first font entry is a space
1072        char c = (*str++) - 32 ;
1073        char_count++ ;
1074        for (int i=0; i<8; i++ ) {
1075          line = pgm_read_byte(TinyFont+((int)c*8)+i);
1076          // pack two pixels per byte using the older color layout
1077          // select each pixel pair from the lookup table
1078            *(draw_loc+i*320) =   pix_value[(line>>6) & 0x03] ;
1079            *(draw_loc+i*320+1) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
1080            *(draw_loc+i*320+2) = pix_value[(line>>2) & 0x03] ;
1081            *(draw_loc+i*320+3) = pix_value[(line) & 0x03] ;
1082        }
1083        draw_loc += 4 ;
1084        x += 8 ;
1085      }
1086      return char_count ;
1087    }
1088
1089    //  vga437 from code block 437 ibm font 1982
1090    // legacy packed-color text, sixteen rows per character
1091    int drawTextVGA437(short x, short y, char * str, char color, char bgcolor){
1092      int char_count = 0 ;
1093      // locate the starting buffer position
1094      char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
1095      // error check
1096      if(x<0 | y<0 | x>630 | y>463) return 0;
1097      // precompute the four foreground and background pixel pairs
1098      char pix_value[4] =
1099        {(bgcolor<<4 | bgcolor), (color<<4 | bgcolor), (bgcolor<<4 | color), (color<<4 | color)
             };
1100      // one row of the character bitmap
1101      unsigned char line;
1102      while (*str){
1103        if((x+8 > 639)) return char_count ;
1104        char c = *str++ ;
1105        char_count++ ;
1106        for (int i=0; i<16; i++ ) {
1107          line = pgm_read_byte(bigFont+((int)c*16)+i);
1108          // pack two pixels per byte using the older color layout
1109          // select each pixel pair from the lookup table
```

```
        *(draw_loc+i*320) =   pix_value[(line>>6) & 0x03] ;
        *(draw_loc+i*320+1) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
        *(draw_loc+i*320+2) = pix_value[(line>>2) & 0x03] ;
        *(draw_loc+i*320+3) = pix_value[(line) & 0x03] ;
    }
    draw_loc += 4 ;
    x += 8 ;
  }
  return char_count ;
}
//
// 16 by 24 font written directly into the older 4-bit buffer layout
// two pixels per byte, unlike the monochrome drawing routines
// http://www.rinkydinkelectronics.com/r_fonts.php
int drawTextArial24(short x, short y, char * str, char color, char bgcolor){
  int char_count = 0 ;
  // locate the starting buffer position
  char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
  // error check
  if(x<0 | y<0 | y>455 ) return 0;
  // precompute the four foreground and background pixel pairs
  char pix_value[4] =
    {(bgcolor<<4 | bgcolor), (color<<4 | bgcolor), (bgcolor<<4 | color), (color<<4 | color)
        };
  // one row of the character bitmap
  unsigned short line;
  while (*str){
    if((x+16 > 639)) return char_count ;
    // font starts at the space character
    char c = (*str++) - 32 ;
    char_count++ ;
    for (int i=0; i<24; i++ ) {
      line = pgm_read_byte(Arial_round_16x24+((int)c*48)+2*i);
      // pack two pixels per byte using the older color layout
      // select each pixel pair from the lookup table
        *(draw_loc+i*320) =   pix_value[(line>>6) & 0x03] ;
        *(draw_loc+i*320+1) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
        *(draw_loc+i*320+2) = pix_value[(line>>2) & 0x03] ;
        *(draw_loc+i*320+3) = pix_value[(line) & 0x03] ;
      line = pgm_read_byte(Arial_round_16x24+((int)c*48)+2*i+1);
      // pack two pixels per byte using the older color layout
      // select each pixel pair from the lookup table
        *(draw_loc+i*320+4) = pix_value[(line>>6) & 0x03] ;
        *(draw_loc+i*320+5) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
        *(draw_loc+i*320+6) = pix_value[(line>>2) & 0x03] ;
        *(draw_loc+i*320+7) = pix_value[(line) & 0x03] ;
    }
    draw_loc += 8 ;
    x += 16 ;
  }
  return char_count ;
}
// grotesk16x32
// http://www.rinkydinkelectronics.com/r_fonts.php
// legacy packed-color text, drawing 31 rows from each 32-row glyph
int drawTextGrotesk32(short x, short y, char * str, char color, char bgcolor){
  int char_count = 0 ;
  // locate the starting buffer position
  char * draw_loc = (current_draw_buffer + ((640 * y + x) >> 1)) ;
  // error check
  if(x<0 | y<0 | y>479-32 ) return 0 ; //(x+16*strlen(str)>639)
  // precompute the four foreground and background pixel pairs
  char pix_value[4] =
    {(bgcolor<<4 | bgcolor), (color<<4 | bgcolor), (bgcolor<<4 | color), (color<<4 | color)
        };
  // one row of the character bitmap
  unsigned short line;
  while (*str){
```

```
1176      // font starts at the space character
1177      if((x+16 > 639)) return char_count ;
1178      char c = (*str++) - 32 ;
1179      char_count++ ;
1180      for (int i=0; i<31; i++ ) {
1181        line = pgm_read_byte(Grotesk16x32+((int)c*64)+2*i);
1182        // pack two pixels per byte using the older color layout
1183        // select each pixel pair from the lookup table
1184          *(draw_loc+i*320) =   pix_value[(line>>6) & 0x03] ;
1185          *(draw_loc+i*320+1) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
1186          *(draw_loc+i*320+2) = pix_value[(line>>2) & 0x03] ;
1187          *(draw_loc+i*320+3) = pix_value[(line) & 0x03] ;
1188        line = pgm_read_byte(Grotesk16x32+((int)c*64)+2*i+1);
1189        // pack two pixels per byte using the older color layout
1190        // select each pixel pair from the lookup table
1191          *(draw_loc+i*320+4) = pix_value[(line>>6) & 0x03] ;
1192          *(draw_loc+i*320+5) = pix_value[(line>>4) & 0x03] ; //draw_loc+i*320
1193          *(draw_loc+i*320+6) = pix_value[(line>>2) & 0x03] ;
1194          *(draw_loc+i*320+7) = pix_value[(line) & 0x03] ;
1195      }
1196      draw_loc += 8 ;
1197      x +=16 ;
1198    }
1199    return char_count ;
1200  }
1201  // =====================================================
1202  // legacy routine
1203  // legacy bold text, drawn twice with a one-pixel offset
1204  void drawBoldTextGLCD(short x, short y, char * str, char textcolor, char textbgcolor, char
      size){
1205     char temp_bg ;
1206      temp_bg = textbgcolor;
1207      while (*str){
1208          char c = *str++;
1209          drawChar(x, y, c, textcolor, textbgcolor, size);
1210          drawChar(x+1, y, c, textcolor, textcolor, size);
1211          x += 7 * size ;
1212      }
1213      textbgcolor = temp_bg ;
1214  }
1215
1216  // ////////////////////////////////////////////
1217  // clear routines with screen clipping
1218  // rectangle clearing uses clipped horizontal lines
1219  // row clearing clamps the requested vertical range
1220  // zero clears to black, any other value fills white
1221  void clearRect(short x1, short y1, short x2, short y2, short c)
1222  {
1223      // x2 and y2 are exclusive: they are not cleared
1224      int top = y1 < 0 ? 0 : y1;
1225      int bottom = y2 > VGA_MONO_HEIGHT ? VGA_MONO_HEIGHT : y2;
1226
1227      for (int y = top; y < bottom; y++) {
1228          drawHLine(x1, y, (int)x2 - x1, c);
1229      }
1230  }
1231  //
1232  // fill from the clipped starting row to the bottom of the screen
1233  void clearLowFrame(short top, short c)
1234  {
1235      int first_row = top;
1236
1237      if (first_row < 0)
1238          first_row = 0;
1239
1240      if (first_row >= VGA_MONO_HEIGHT)
1241          return;
1242
```

```
    memset(current_draw_buffer +
              first_row * VGA_BYTES_PER_ROW, c != 0 ? 0xFF : 0x00,
              (VGA_MONO_HEIGHT - first_row) * VGA_BYTES_PER_ROW);
}

// region from y1 to y2 with y1 < y2
void clearRegion(short y1, short y2, short c)
{
    // clear rows y1 through y2 - 1
    int top = y1 < 0 ? 0 : y1;
    int bottom = y2 > VGA_MONO_HEIGHT ? VGA_MONO_HEIGHT : y2;

    if (bottom <= top)
        return;

    memset(current_draw_buffer + top * VGA_BYTES_PER_ROW, c != 0 ? 0xFF : 0x00,
           (bottom - top) * VGA_BYTES_PER_ROW);
}
// ====================================
// buffer copy utilities
#ifndef DOUBLE_BUFFER_NONE
  void copy_buffer0to1(void){
    memcpy(vga_buffer_1, vga_buffer_0, VGA_BUFFER_COUNT) ;
  }

  void copy_buffer1to0(void){
    memcpy(vga_buffer_0, vga_buffer_1, VGA_BUFFER_COUNT) ;
  }

  // copy the current drawing buffer into the other buffer
  void copy_buffer_to_other(void) {
      if((int)current_draw_buffer == (int)vga_buffer_1)
        memcpy(vga_buffer_0, vga_buffer_1, VGA_BUFFER_COUNT) ;
      else
        memcpy(vga_buffer_1, vga_buffer_0, VGA_BUFFER_COUNT) ;
  }
#endif

// ====================================
// driver communication with thread
// consume a start flag of 1, leave other values unchanged
int draw_start_signal(void){
  if(start_flag==1) {
    start_flag = 0 ;
    return 1 ;
  }
  else {
    return 0 ;
  }
}

// returns 1 for 60fps, 2 for 30fps, 3 for no double buffer
int get_buffer_type(void){
  return buffer_type ;
}

////////////////////////////////////////////////
// read back from vga
// get the color of a pixel
// read from the current drawing buffer
short readPixel(short x, short y)
{
    if (x < 0 || x >= VGA_MONO_WIDTH ||
        y < 0 || y >= VGA_MONO_HEIGHT)
        return BLACK;

    const uint8_t *buffer =
        (const uint8_t *)current_draw_buffer;
```

```
1311
1312     int index = y * VGA_BYTES_PER_ROW + (x >> 3);
1313     uint8_t mask = (uint8_t)(1u << (x & 7));
1314
1315     return (buffer[index] & mask) ? WHITE : BLACK;
1316 }
1317
1318 ////////////////////////////////////////////////
1319 // draw a small cross extending two pixels from the center
1320 void crosshair(short x, short y, short c){
1321   drawPixel(x,y,c);
1322   drawPixel(x-1,y,c);
1323   drawPixel(x+1,y,c);
1324   drawPixel(x,y-1,c);
1325   drawPixel(x,y+1,c);
1326   drawPixel(x-2,y,c);
1327   drawPixel(x+2,y,c);
1328   drawPixel(x,y-2,c);
1329   drawPixel(x,y+2,c);
1330 }
1331
1332 ////////////////////////////////////////////////
```

A.3 VGA Header: `VGA/vga16_graphics_v3.h`

Listing 3: VGA declarations and shared definitions.

```
1  /**
2   * hunter adams (vha3@cornell.edu)
3   * modifed for 16 colors by brl4
4   *
5   *original color driver aimed at rp2350, current buffers use monochrome data
6
7   * hardware connections
8   *   - gpio 16 ---> vga hsync
9   *   - gpio 17 ---> vga vsync
10  *   - gpio 18 ---> 470 ohm resistor ---> vga green
11  *   - gpio 19 ---> 330 ohm resistor ---> vga green
12  *   - gpio 20 ---> 330 ohm resistor ---> vga blue
13  *   - gpio 21 ---> 330 ohm resistor ---> vga red
14  *   - rp2040 gnd ---> vga gnd
15  *
16  * resources used
17  *   - pio state machines 0, 1, and 2 on pio instance 0
18  *   - 4 dma channels
19  *   - 2 x 38.4 kbytes of ram for the current monochrome buffers
20  *
21  */
22
23
24 // names for the video output pins
25  enum vga_pins {HSYNC=16, VSYNC, LO_GRN, HI_GRN, BLUE_PIN, RED_PIN} ;
26
27 // legacy 16-color names, monochrome drawing treats any nonzero color as white
28 enum colors {BLACK, DARK_GREEN, MED_GREEN, GREEN,
29              DARK_BLUE, BLUE, LIGHT_BLUE, CYAN,
30              RED, DARK_ORANGE, ORANGE, YELLOW,
31              MAGENTA, PINK, LIGHT_PINK, WHITE} ;
32
33 // cell drawing and neighbor checks for a grid of 2 by 2 pixel cells
34 void drawCell(short x, short y, char color) ;
35 int checkNeighbors(short x, short y) ;
36 int isAlive(short x, short y) ;
37
38
```

```
39 // set up video output before drawing
40 void initVGA(void) ;
41
42 // ========================
43 // dma signals for buffer timing
44 // consume a draw-start flag of 1, return zero for other values
45 int draw_start_signal(void);
46 // buffer mode: 1 for 60 fps, 2 for 30 fps, 3 for a single buffer
47 int get_buffer_type(void) ;
48
49 // ========================
50 // shapes and fills
51 void drawPixel(short x, short y, char color) ;
52 void drawVLine(short x, short y, short h, char color) ;
53 void drawHLine(int x, int y, int w, char color) ; // clipped line with whole-byte fills for
       speed
54 void drawLine(short x0, short y0, short x1, short y1, char color) ;
55 void drawRect(short x, short y, short w, short h, char color);
56 void drawCircle(short x0, short y0, short r, char color) ;
57 void drawCircleHelper( short x0, short y0, short r, unsigned char cornername, char color) ;
58 void fillCircle(short x0, short y0, short r, char color) ;
59 void fillCircleHelper(short x0, short y0, short r, unsigned char cornername, short delta,
       char color) ;
60 void drawRoundRect(short x, short y, short w, short h, short r, char color) ;
61 void fillRoundRect(short x, short y, short w, short h, short r, char color) ;
62 void fillRect(short x, short y, short w, short h, char color) ;
63 void fillTri(float x0, float y0, float x1, float y1, float x2, float y2, char color) ;
64 void drawMultiLine(int num_lines,  short point_list[][2], char color) ;
65 // ====================
66 // text routines with local cursor state
67 // glcd supports any x, other fonts still use the older packed-color layout
68 int drawTextGLCD(short x, short y, char * string, char color, char bakgnd_color);
69 int drawTextAscii(short x, short y, char * str, char color, char bgcolor);
70 int drawTextVGA437(short x, short y, char * string, char color, char bakgnd_color);
71 int drawTextArial24(short x, short y, char * str, char color, char bgcolor);
72 int drawTextTiny8(short x, short y, char * str, char color, char bgcolor) ;
73 int drawTextGrotesk32(short x, short y, char * str, char color, char bgcolor) ;
74 // ====================
75 //
76 // ====================
77 // specialized clear routines
78 // clipped rectangle clear, excluding the x2 and y2 edges
79 void clearRect(short x1, short y1, short x2, short y2, short c) ;
80 // clear from the starting row through the bottom of the frame
81 void clearLowFrame(short, short) ;
82 // clear rows from y1 up to, but not including, y2
83 void clearRegion(short y1, short y2, short c) ;
84
85 // ====================
86 // buffer handling
87 // copy static content into both buffers to keep it visible after swaps
88 // copy buffer 0 to 1 and 1 to 0
89 void copy_buffer0to1(void) ;
90 void copy_buffer1to0(void) ;
91 // update the other buffer with the latest drawing
92 // copy from the current draw buffer
93 void copy_buffer_to_other(void) ;
94
95 // ====================
96 // pixel reading and crosshair drawing
97 // read a pixel from the current drawing buffer
98 short readPixel(short, short) ;
99
100 // draw a small cross around the center positon
101 void crosshair(short x, short y, short c) ;
102
103 // ================================================
104 // legacy text helpers using shared settings
```

```
// older character and string routines
// use the glcd text routine above for the monochrome buffer
void drawChar(short x, short y, unsigned char c, char color, char bg, unsigned char size) ;
void setCursor(short x, short y);
void setTextColor(char c);
void setTextColor2(char c, char bg);
void setTextSize(unsigned char s);
void setTextWrap(char w);
void tft_write(unsigned char c) ;
void writeString(char* str) ;
// ===  strings added 10/11/2023 brl4
void drawCharBig(short x, short y, unsigned char c, char color, char bg) ;
void writeStringBig(char* str) ;
void setTextColorBig(char, char); //stores foreground and backgound colors like the usual setter
// bold text made by drawing each character twice with a small offset
void writeStringBold(char* str);
void drawBoldTextGLCD(short x, short y, char * str, char textcolor, char textbgcolor, char size);
// !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

A.4 Horizontal Sync: `VGA/hsync.pio`

Listing 4: Horizontal synchronization PIO program and initialization.

```
;
; hunter adams (vha3@cornell.edu)
; horizontal sync timing for the vga driver


; pio program name
.program hsync

; front porch: 16 clocks at 25 mhz
; sync pulse: 96 clocks
; back porch: 48 clocks, including the next mov instruction
; active display: 640 clocks
;
; high for 704 cycles, or 28.16 us
; low for 96 cycles, or 3.84 us
; full line: 800 cycles, or 32 us
;


pull block              ; load the loop count into osr once
.wrap_target            ; repeat from here for each line

; active display and front porch
mov x, osr              ; reload the countdown for this line
activeporch:
   jmp x-- activeporch  ; count down while the sync pin stays high

; sync pulse
pulse:
    set pins, 0 [31]    ; hold low for 32 cycles
    set pins, 0 [31]    ; keep low, 64 cycles total
    set pins, 0 [31]    ; keep low, 96 cycles total

; back porch
backporch:
    set pins, 1 [31]    ; hold high for 32 cycles
    set pins, 1 [12]    ; keep high, 45 cycles total
    irq 0       [1]     ; signal the other state machines, 47 cycles total
.wrap

```

```
41
42 % c-sdk {
43 #include "hardware/clocks.h"
44 static inline void hsync_program_init(PIO pio, uint sm, uint offset, uint pin) {
45
46     // get the generated default settings for this program
47     pio_sm_config c = hsync_program_get_default_config(offset);
48
49     // map set instructions to the single hsync output pin
50     sm_config_set_set_pins(&c, pin, 1);
51
52     // calculate the divider from the current system clock
53     // keep the state machine running at 25 mhz
54     sm_config_set_clkdiv(
55         &c,
56         (float)clock_get_hz(clk_sys) / 25000000.0f
57     );
58     // connect pio output to the gpio pin
59     pio_gpio_init(pio, pin);
60     // pio_gpio_init(pio, pin+1);
61
62     // set the pin direciton to output
63     pio_sm_set_consecutive_pindirs(pio, sm, pin, 1, true);
64
65     // apply the settings and start at the program offset
66     pio_sm_init(pio, sm, offset, &c);
67
68     // leave disabled for now, started togther with the other state machines
69     // pio_sm_set_enabled(pio, sm, true);
70 }
71 %}
```

A.5 Vertical Sync: `VGA/vsync.pio`

Listing 5: Vertical synchronization PIO program and initialization.

```
1  ;
2  ; Hunter Adams (vha3@cornell.edu)
3  ; VSync generation for the VGA driver.
4  ;
5  ; HSync sets IRQ 0 once per scanline. This program counts those
6  ; events to generate the active region and vertical blanking.
7  ; During active lines, IRQ 1 tells the RGB state machine to output
8  ; one scanline of pixel data.
9  ;
10
11 .program vsync
12 .side_set 1 opt               ; Optional one-bit side-set controls VSync
13
14 ; Vertical frame layout:
15 ;   Active region: 480 lines when C supplies 479
16 ;   Front porch:    10 lines
17 ;   Sync pulse:      2 lines
18 ;   Back porch:     32 lines
19 ;
20 ; This version omits the usual 33rd back-porch line.
21
22 pull block                   ; Load the active-line counter from the TX FIFO
23                              ; into OSR; this happens only at startup
24
25 .wrap_target
26
27 ; ACTIVE DISPLAY REGION
28 mov x, osr                   ; Reload the active-line counter each frame
29
```

```
activefront:
    wait 1 irq 0             ; Wait for the next HSync line event;
                             ; WAIT clears the IRQ flag when satisfied
    irq 1                    ; Signal an active scanline to the RGB program
    jmp x-- activefront      ; Repeat for X + 1 lines

; FRONT PORCH
set y, 9                     ; Ten iterations: counter values 9 through 0

frontporch:
    wait 1 irq 0             ; Wait for one scanline without enabling RGB
    jmp y-- frontporch

; VERTICAL SYNC PULSE
set pins, 0                  ; Assert the active-low VSync signal
wait 1 irq 0                 ; First sync line
wait 1 irq 0                 ; Second sync line

; BACK PORCH
set y, 31                    ; Thirty-two iterations: values 31 through 0

backporch:
    wait 1 irq 0 side 1      ; Deassert VSync while waiting for a line event
    jmp y-- backporch         ; Keep RGB blank throughout the back porch

.wrap                        ; Return to the next frame's active region


% c-sdk {
#include "hardware/clocks.h"

static inline void vsync_program_init(PIO pio, uint sm,
                                      uint offset, uint pin)
{
    // Start with the generated configuration for this PIO program.
    // It includes the wrap addresses and side-set configuration.
    pio_sm_config c = vsync_program_get_default_config(offset);

    // Both SET instructions and side-set control the VSync GPIO.
    sm_config_set_set_pins(&c, pin, 1);
    sm_config_set_sideset_pins(&c, pin);

    // Keep the state-machine instruction clock at approximately 25 MHz.
    // Calculate the divider from the actual system clock so that
    // changing the CPU frequency does not proportionally speed up PIO.
    // Vertical line timing itself is driven by HSync's IRQ 0 events.
    sm_config_set_clkdiv(
        &c,
        (float)clock_get_hz(clk_sys) / 25000000.0f
    );

    // Connect the GPIO to this PIO instance and configure it as an output.
    pio_gpio_init(pio, pin);
    pio_sm_set_consecutive_pindirs(pio, sm, pin, 1, true);

    // Apply the configuration and set the initial program counter.
    // Initialization leaves the state machine disabled.
    pio_sm_init(pio, sm, offset, &c);

    // The VGA driver supplies the counter and starts the HSync,
    // VSync, and RGB state machines together.
}
%}
```

A.5.1 Monochrome Output: `VGA/rgb.pio`

Listing 6: Monochrome pixel-output PIO program and initialization.

```
‘‘‘pio
;
; Monochrome VGA pixel output.
; Each framebuffer byte stores eight pixels, least-significant bit first.
; A zero bit produces black; a one bit drives all four color pins high.
;
; Before starting, the VGA driver loads ISR with 639 for 640 pixels
; per scanline and primes OSR for the first automatic FIFO pull.
;

.program rgb
.define PUBLIC vid_freq 150  ; Target state-machine frequency in MHz

.wrap_target
    mov x, isr [1]          ; Reload the pixel counter; two cycles also
                            ; complete the previous line's final pixel hold
    set pins, 0             ; Drive black during horizontal/vertical blanking
    wait 1 irq 1 [2]        ; Wait for VSync's active-line signal, clear it,
                            ; then delay two cycles before reading pixels

pixel:
    out y, 1                ; Shift the next framebuffer bit into Y
    jmp !y black            ; Choose the black path when the bit is zero

    set pins, 15 [1]        ; Drive all four color pins high; hold two cycles
    jmp next_pixel          ; Skip the black-output instruction

black:
    set pins, 0 [2]         ; Drive all four color pins low; hold three cycles

next_pixel:
    jmp x-- pixel           ; Repeat for X + 1 pixels: 640 when X starts at 639
.wrap                       ; Return to blanking after the last pixel


% c-sdk {
#include "hardware/clocks.h"

static inline void rgb_program_init(PIO pio, uint sm,
                                    uint offset, uint pin)
{
    // Load the generated configuration, including the program's
    // wrap addresses.
    pio_sm_config c = rgb_program_get_default_config(offset);

    // SET instructions control four consecutive VGA color pins,
    // beginning at the supplied GPIO.
    sm_config_set_set_pins(&c, pin, 4);

    // Shift OSR right so pixels are consumed least-significant bit first.
    // Enable automatic pulling with an eight-bit threshold.
    // Each FIFO entry supplies eight pixel bits before another is pulled.
    sm_config_set_out_shift(&c, true, true, 8);

    // Combine the TX and RX FIFO storage into a larger TX-only FIFO
    // to provide more buffering for incoming pixel data.
    sm_config_set_fifo_join(&c, PIO_FIFO_JOIN_TX);

    // Both black and white paths take six instruction cycles per pixel.
    // A 150 MHz state-machine clock therefore gives a nominal
    // 25 MHz pixel rate. Derive the divider from the actual system
    // clock to preserve pixel timing when the CPU is overclocked.
    sm_config_set_clkdiv(
        &c,
        (float)clock_get_hz(clk_sys) / 150000000.0f
    );
```

```
67
68      // Connect each of the four color GPIOs to this PIO instance.
69      for (uint i = 0; i < 4; ++i) {
70          pio_gpio_init(pio, pin + i);
71      }
72
73      // Configure all four color pins as outputs.
74      pio_sm_set_consecutive_pindirs(pio, sm, pin, 4, true);
75
76      // Apply the configuration and set the initial program counter.
77      // Initialization leaves the state machine disabled.
78      pio_sm_init(pio, sm, offset, &c);
79
80      // The VGA driver prepares the pixel counter and FIFO/DMA path,
81      // then starts the HSync, VSync, and RGB state machines together.
82  }
83  %}
84  ```
```